



AC638N 用户手册

珠海市杰理科技股份有限公司

Zhuhai Jieli Technologyco.,LTD

版权所有，未经许可，禁止外传

2021 年 6 月

修改记录

版本	更新日期	描述
V1.0	2021-6-9	初版

珠海市杰理科技股份有限公司

目录

AC638N 用户手册.....	1
序言.....	6
第 1 章. 总体介绍.....	7
1.1. 概述.....	7
1.2. 系统框图.....	9
第 2 章. IO_CONTROL.....	10
2.1. IOMC.....	10
2.1.1. 概述.....	10
2.1.2. 控制寄存器.....	10
2.2. 引脚通用功能表.....	13
2.3. PORT Control Crossbar.....	15
2.3.1. Crossbar 功能结构图.....	15
2.3.2. 寄存器功能.....	17
第 3 章. 32 位定时器(TIMER32).....	18
3.1. 概述.....	18
3.2. 控制寄存器.....	18
3.3. 寄存器说明.....	18
第 4 章. ADC.....	22
4.1. 概述.....	22
4.2. 寄存器说明.....	22
第 5 章. IIC.....	27
5.1. 概述.....	27
5.2. 寄存器说明.....	27
第 6 章. 红外过滤（IRFLT）.....	35
6.1. 概述.....	35
6.2. 寄存器说明.....	37
6.3. 例程.....	38
第 7 章. PULSE COUNTER.....	40

7.1. 概述.....	40
7.2. 寄存器说明.....	40
7.3. 示例代码.....	41
第 8 章. RDEC.....	43
8.1. 概述.....	43
8.2. 控制寄存器.....	43
第 9 章. SPI.....	45
9.1. 概述.....	45
9.2. 控制寄存器.....	46
9.3. 传输波形.....	50
第 10 章. UART.....	51
10.1. 概述.....	51
10.2. 控制寄存器.....	51
第 11 章. USB BRIDGE.....	57
11.1. 概述.....	57
11.2. 寄存器说明.....	58
第 12 章. SD 控制器.....	62
12.1. 概述.....	62
12.2. 控制寄存器.....	62
12.3. 模块寄存器.....	62
12.4. 配置流程示例.....	69
第 13 章. PWM_LED.....	72
13.1. 概述.....	72
13.2. 特性.....	72
13.3. 寄存器并接.....	73
13.4. 时钟控制.....	73
13.5. 二级亮灭控制.....	74
13.6. 呼吸灯控制.....	76
13.7. 3.3v 系统控制寄存器说明.....	77

13.8. 软件 DEMO.....	83
--------------------	----

珠海市杰理科技股份有限公司

序言

AC638N 系列芯片是集成三模蓝牙的系统级 SOC，支持高性能低功耗的蓝牙应用。芯片内置蓝牙调制解调器、基带及模拟 RF 模块，支持经典蓝牙（BR/EDR）和低功耗蓝牙（LE），v5.1 版本及低功耗蓝牙连接待机，通信距离长，待机时间久。AC638N 内置 32 位双发射 DSP，支持单精度浮点，支持在线调试，支持数学运算加速的协处理器，整合多种硬件加速模块，方便开发。支持 SDcard、USB、FLASH 等多个外设。

AC638N 主推 BluetoothSmart 和 SmartReady 应用方案，并提供了蓝牙双模的单芯片解决方案，减少设计的工作量让产品更快赢得市场。

AC638N 集成了低成本、超低功耗、2.4G 射频模块，并提供极低的射频活动功耗和 MCU 功耗，支持低功耗模式，出色的电池使用寿命使其适合功耗敏感的应用。

AC638N 提供了完整的 FCC 与 BQB 的认证，使客户在该系列下开发自己的产品时减少了认证的费用。

AC638N 集成了一整套的蓝牙解决方案，为用户开发提供了方便的环境，为产品提供了无限的可能。

第 1 章. 总体介绍

1.1. 概述

CPU

- ❖ 32 位双发射 DSP，最高 160MHz
- ❖ 支持 IEEE754 单精度浮点
- ❖ 支持 cordic 加速引擎
- ❖ 支持在线单步调试，支持内核寄存器监控，内核总线监控，拥有 8 个硬断点
- ❖ 支持系统异常捕获分析单元

Memory

- ❖ 片上集成 176K 字节 SRAM
- ❖ 片上集成 64K 字节 Maskrom
- ❖ 32K ICache, 4K DCache (只读数据)
- ❖ SRAM 支持 MMU (页大小 128byte)
- ❖ MPU 支持 5 种权限管理 (内核可读可写可执行, 外设可读可写)

数学运算

- ❖ 支持 2^N (32-1024) 点复数、实数 FFT/IFFT
- ❖ AES128
- ❖ SBC 编解码

数字设备

- ❖ 蓝牙双模基带，支持 v5.1 版本
- ❖ Full Speed USB OTG 控制器，除 EP0 外带 4 个 In EP、4 个 Out EP

中断控制器

- ❖ 64 个中断源，8 级可编程中断优先级
- ❖ 支持多优先级不可屏蔽中断
- ❖ 支持低功耗轮询唤醒机制
- ❖ 支持 8 个外部 I/O 中断 (低功耗可用，可复用到任意 IO，带硬件滤波)
- ❖ 带软中断功能，优先级可以配置

数字 I/O

- ❖ 最多 27 个可编程数字 I/O 引脚
- ❖ 支持 Crossbar 重映射 IO 功能
- ❖ 不同的功能引脚有不同的电流选择，最大拉/灌电流为 64mA (普通 IO) / 8mA (耐高压 IO)
- ❖ 可配置上拉 10K、下拉 10K 功能

- ❖ 6 个 32 位异步分频可重载定时器，可用作计时、捕捉，PWM 模式
- ❖ 硬件看门狗 (Watchdog)
- ❖ 3 个串口控制器 (UART0/1/2)，支持 DMA，其中 UART1 支持流控制
- ❖ 1 个 IIC 控制器，支持主机模式和从机模式
- ❖ 3 个 SPI 控制器，支持 1/2/4 线，支持 DMA
- ❖ 1 个 SPI FLASH 控制器 (SFC)，支持运行程

序和作为可读数据空间

- ❖ 1 个 SD host 控制器，支持 1/4 线，支持 DMA
- ❖ 独立推灯模块，支持呼吸灯，可在正常及低功耗模式下工作
- ❖ 一个旋转编码器 RDEC
- ❖ 红外遥控检测模块
- ❖ 低功耗脉冲计数器（累计走路步数）

模拟模块

- ❖ 蓝牙收发 RF 前端
- ❖ USB 1.1 full speed PHY
- ❖ 10 位精度 16 通道 ADC（Key/电压检测等）
- ❖ DCDC/LDO，输出 typ=1.25v
- ❖ LVD，支持掉电保护
- ❖ PMU，支持软开关功能，蓝牙 sniff 低功耗，可独立供电
- ❖ 锂电池充电模块
- ❖ 充电脚支持与充电仓通信，支持充电仓进出
- ❖ 低功耗唤醒
- ❖ 系统 PLL，中心频率为 192MHz

❖ RC，内部 RC 时钟，频率在 16MHz 左右

❖ LRC，内部低温漂 RC 时钟，频率在 32KHz 左右

❖ 支持 12~40M 外部晶振

❖ LP_CTMU 低功耗高精度触摸按键模块，支持 2 路输入

❖ CTMU 高精度触摸按键模块，支持 8 路输入

❖ 64 位 EFUSE

工作范围

❖ 工作电压

VPWR: 4.5V~6.5V

VBAT: 2.7V~4.5V

IOVDD: 2.2V~3.3V

❖ 工作温度（-40° C 至+85° C）

封装

❖ QFN32 QSOP24

❖ 裸片

1.2. 系统框图

AC638N 系统框图如下：

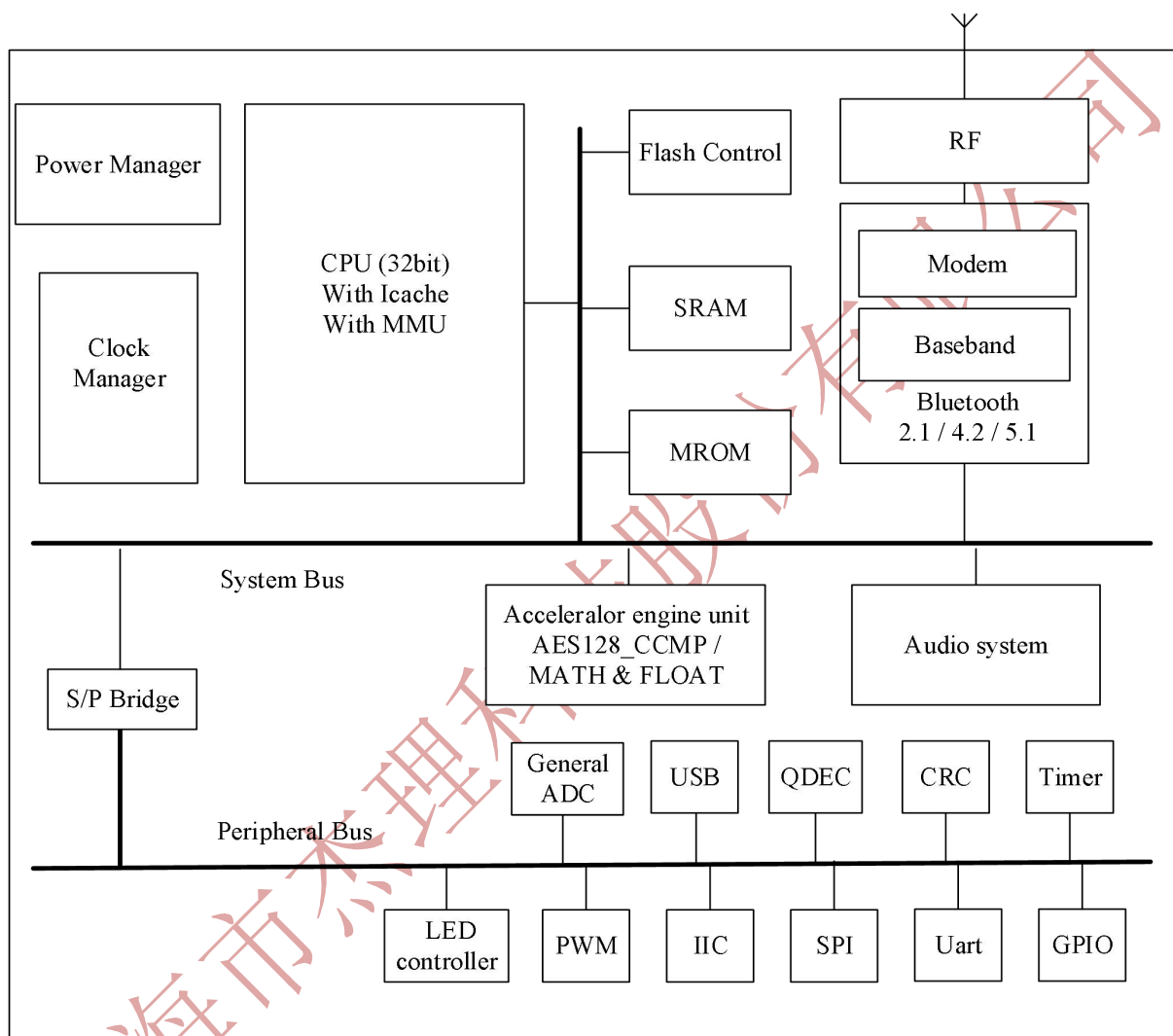


图 1-1 AC638N 系统框图

第 2 章. IO_CONTROL

2.1. IOMC

2.1.1. 概述

IOMC 控制寄存器主要用于控制 IO 除 crossbar 之外的一些拓展功能的配置。

2.1.2. 控制寄存器

1.IOMC0

Name	Bit	Default	Description						
sfc_ios	IOMC0[16]	0	SFC 模块 IO 选择						
			sel	CS	CK	D0	D1	D2	D3
			0	PD3	PD0	PD1	PD2	PB4	PC3
			1	PD0	PD6	PD7	PD1	PC5	PA5
sdtap_ios	IOMC0[18:17]	0		00	01	10	11		
			CK	PC4	USBDP	PB1	PB6		
			DA	PC5	USBDM	PB2	PB7		
spi0_mix_mode	IOMC0[19]	0	SPI0 输入结果产生选项 0: SPI0 内部输入为 DI; 1: SPI0 内部输入为 DI&DO;						
cap_mux_edge	IOMC0[20]	0	input channel 2 的 CAP 边沿选择（用于 GPCNT、PCNT） 0: 上升沿; 1: 下降沿;						

2.IOMC1

Name	Bit	Default	Description
------	-----	---------	-------------

wkup_ich_sel	IOMC1[3:0]	0	输入选择: 0: GP_ICH0 1: GP_ICH1 2: GP_ICH2 3: GP_ICH3 4: GP_ICH4 5: GP_ICH5 6: GP_ICH6 7: GP_ICH7 8: TMR0_PWM 9: TMR1_PWM
irflt_ich_sel	IOMC1[7:4]	0	输入选择: (同 wkup_ich_sel)
cap_ich_sel	IOMC1[11:8]	0	输入选择: (同 wkup_ich_sel)
clk_ich_sel	IOMC1[15:12]	0	输入选择: (同 wkup_ich_sel)
wl_ich_sel	IOMC1[19:16]	0	输入选择: (同 wkup_ich_sel)

3.IOMC2

Name	Bit	Default	Description
gp_och0_sel	IOMC2[4:0]	0	输出选择： 0: PMU_DBG 1: CLK_OUT0 2: CLK_OUT1 3: CLK_OUT2 4: CLK_OUT3 5: WLC_INT_ACT 6: WLC_INT_STA 7: WLC_INT_FRQ 8: WL_DBG0 9: WL_DBG1 10: WL_DBG2 11: WL_DBG3 12: WL_DBG4 13: WL_DBG5 14: WL_DBG6 15: WL_DBG7 16: DCDC_TEST0 17: DCDC_TEST1
gp_och1_sel	IOMC2[9:5]	0	输出选择：（同 gp_och0_sel）
gp_och2_sel	IOMC2[14:10]	0	输出选择：（同 gp_och0_sel）
gp_och3_sel	IOMC2[19:15]	0	输出选择：（同 gp_och0_sel）
gp_och4_sel	IOMC2[24:20]	0	输出选择：（同 gp_och0_sel）
gp_och5_sel	IOMC2[29:25]	0	输出选择：（同 gp_och0_sel）

4.IOMC3

Name	Bit	Default	Description
gp_och6_sel	IOMC3[4:0]	0	输出选择: (同 gp_och0_sel)
gp_och7_sel	IOMC3[9:5]	0	输出选择: (同 gp_och0_sel)
gp_och8_sel	IOMC3[14:10]	0	输出选择: (同 gp_och0_sel)
gp_och9_sel	IOMC3[19:15]	0	输出选择: (同 gp_och0_sel)
gp_och10_sel	IOMC3[24:20]	0	输出选择: (同 gp_och0_sel)
gp_och11_sel	IOMC3[29:25]	0	输出选择: (同 gp_och0_sel)

2.2. 引脚通用功能表

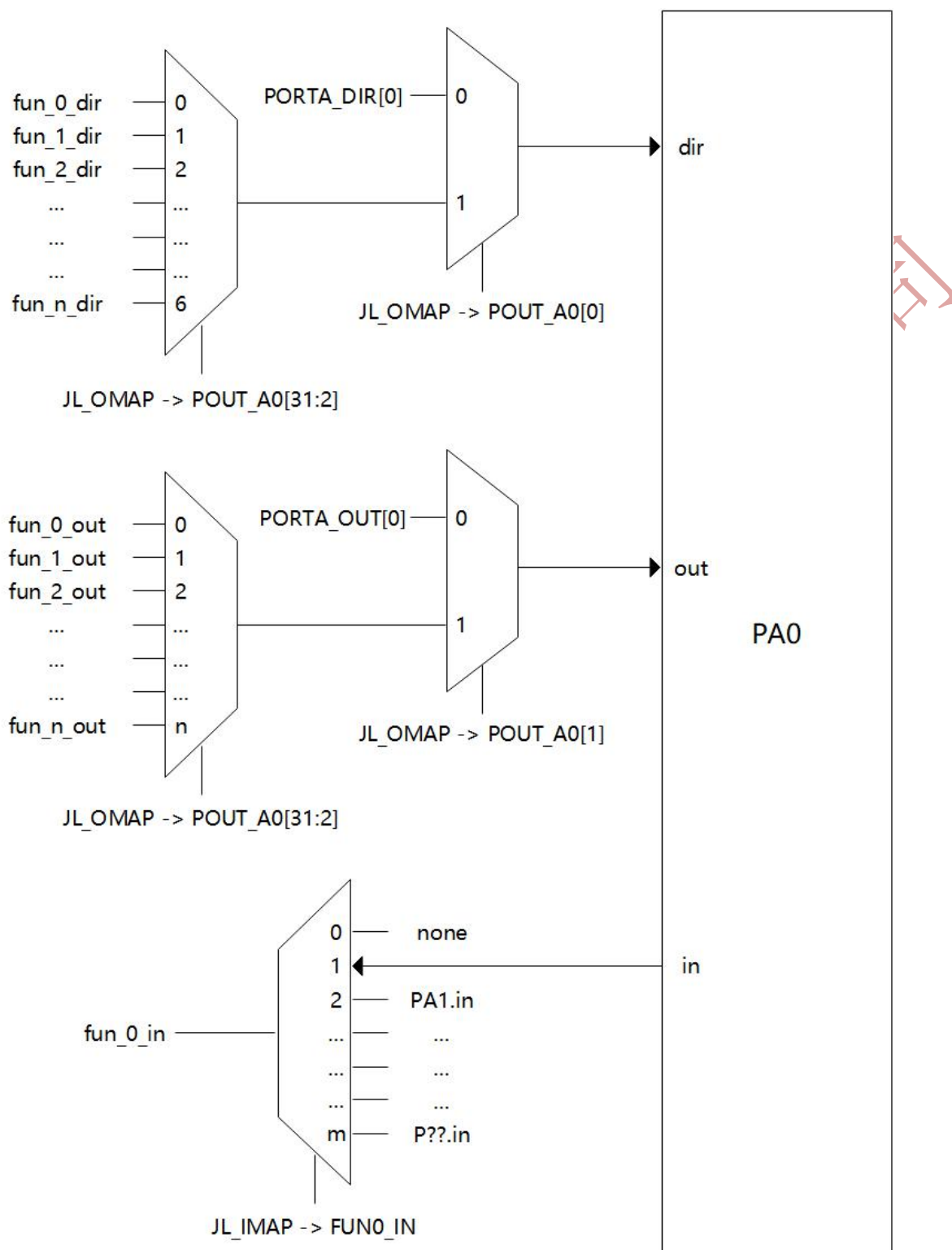
端口											
可复用的功能											
PA1		MICIN0		ISP_DI_B	SPI1DIC					UART1TXC	PWM0
PA2	CLKOUT	MICBIAS0		ISP_CLK_B	SPI1CLKC			ALNK_MC		UART1RXC	CAP3
	1							LKA			
PA3					SPI1DOC			ALNK_DA	AD	Tou	UART2TXA
								T2A	C0	ch0	PWM1
PA4					SPI2DIA			ALNK_DA			UART2RXA
								T3A			CAP2
PA5	UART1_	PLNK_SCLK/		spi0_DAT3	SPI2CLKA	SFC_DAT	IIC_SCL_	ALNK_DA	AD	Tou	UART0TXA
	CTS	ANCCK		B(3)		3B(3)	D	T0AB	C1	ch2	PWM5
PA6	UART1_R	PLNK_DAT0/			SPI2DOA		IIC_SDA	ALNK_DA	AD	Tou	UART0RXA
	TS	ANCDR					_D	T1AB	C2	ch3	CAP0
PB1	SDTAP_C	LP_Touch0	FMTX	默认长按	SPI2CLKC				AD		UART2TXC
(上	LKC		_A	Reset					C6		
拉)											
PB2	SDTAP_	LP_Touch1			SPI2DOC				AD		UART2RXC
											CAP5

	DATC								C7			
PB3		EVDD/PVDD	FMTX _B		SPI2DIC			ALNK_MC LKB			UART1TXB/UA RT1RXB	TMR4
PB4	CLKOUT 0	LVD	SD0D AT0B	spi0_DAT2 A(2)	SPI1DIA	SFC_DAT 2A(2)			AD C12	Tou ch6	UART1TXA	TMR2
PB6	SDTAP_C LKD		SD0C MDB		SPI1CLKA		Q-deco der0		AD C9	Tou ch7	UART1RXA	PWM2
PB7	SDTAP_ DATD		SD0C LKB		SPI1DOA		Q-deco der1		AD C8	Tou ch1	UART0TXB	
PB8											UART0RXB	CAP4
PC2	PA_EN	PLNK_DAT1/ ANCDE			SPI2DIB		IIC_SCL_ C	ALNK_SC LKB		Tou ch4	UART0TXD	TMR1
PC3	LNA_EN		SD0D AT0A	spi0_DAT3 A(3)	SPI1DIB	SFC_DAT 3A(3)	IIC_SDA _C	ALNK_LR CKB	AD C3	Tou ch5	UART0RXD	TMR3
PC4	SDTAP_C LKA		SD0C MDA		SPI1CLKB		IIC_SCL_ B	ALNK_DA T2B	AD C4		UART2TXD	PWM4
PC5	SDTAP_ DATA		SD0C LKA	spi0_DAT2 B(2)	SPI1DOB	SFC_DAT 2B(2)	IIC_SDA _B	ALNK_DA T3B	AD C5		UART2RXD	
PD0 (上 拉)			spi0_ CSB	spi0_CLKA	SFC_CSB	SFC_CLK A						
PD1			spi0_ DIB(1)	spi0_DOA(0)	SFC_DIB(1)	SFC_DO A(0)						
PD2				spi0_DIA(1)		SFC_DIA(1)						
PD3 (上				spi0_CSA		SFC_CSA						

拉)												
PD4(	Flash_A											
输出	供电脚											
0)												
PD5(	Flash_B											
输出	供电脚											
0)												
PD6			spl0_		SFC_CLKB			ALNK_SC			UART2TXB	TMR0
			CLKB					LKA				
PD7			spl0_		SFC_DOB(			ALNK_LR			UART2RXB	TMR5
			DOB(		0)			CKA				
			0)									
USBD	SDTAP_C				SPI2CLKB			IIC_SCL_		AD	UART1TXD	
P(下	LKB							A		C10		
拉)												
UDB	SDTAP_				SPI2DOB			IIC_SDA		AD	UART1RXD	
DM(	DATB							_A		C11		
下拉)												
LDO5											UART0TXC/UA	PWM3/
V											RTORXC	CAP1
P00	8051专											
	用IO											

2.3. PORT Control Crossbar

2.3.1. Crossbar 功能结构图



2.3.2. 寄存器功能

1.JL_OMAP -> POUT

Bit	Name	RW	Description
31-2	FUN_SELECT	rw	Crossbar 输出功能选择： (见功能表或头文件)
1	FUN_OUT_EN	rw	Crossbar 数据控制使能： 0: CPU 数据 1: 外设数据
0	FUN_DIR_EN	rw	Crossbar 方向控制使能： 0: CPU 控制 1: 外设控制 【注意】 ：若所选外设无方向控制功能，则都由 CPU 控制

2.JL_IMAP -> FUN_IN

Bit	Name	RW	Description
31-0	INPUT_SELEC T	rw	Crossbar 输入引脚选择： (见功能表或头文件)

第 3 章. 32 位定时器(TIMER32)

3.1. 概述

Timer32 是一个集合了定时/计数/捕获功能于一体的多动能 32 位定时器。它的驱动源可以选择片内时钟或片外信号。它带有一个可配置的最高达 64 的异步预分频器，用于扩展定时时间或片外信号的最高频率。它还具有上升沿/下降沿捕获功能，可以方便的对片外信号的高电平/低电平宽度进行测量。

3.2. 控制寄存器

寄存器列表	TIMER0	TIMER1	TIMER2	TIMER3	TIMER4	TIMER5
Tx_CON	T0_CON	T1_CON	T2_CON	T3_CON	T4_CON	T5_CON
Tx_CNT	T0_CNT	T1_CNT	T2_CNT	T3_CNT	T4_CNT	T5_CNT
Tx_PRD	T0_PRD	T1_PRD	T2_PRD	T3_PRD	T4_PRD	T5_PRD
Tx_PWM	T0_PWM	T1_PWM	T2_PWM	T3_PWM	T4_PWM	T5_PWM

3.3. 寄存器说明

1.Tx_CON: timer x control register

Bit	Name	RW	Description
31-16	reserved	r	预留
15	PND	r	PND: 中断请求标志，当 timer 溢出或产生捕获动作时会被硬件置 1，需要由软件清 0。
14	PCLR	w	PCLR: 软件在此位写入‘1’将清除 PND 中断请求标志。
13-10	SSEL[3:0]	rw	SSEL3-0: timer 驱动源选择 【注意】 ：选中的时钟需要使用上面 PSET 分频到(lsb_clk/2)以下！

			0: 使用 LSB 时钟作为 timer 的驱动源; 1: 使用 OSC 口信号作为 timer 的驱动源; 2: 使用 HTC 时钟作为 timer 的驱动源; 3: 使用 LRC 时钟作为 timer 的驱动源; 4: 使用 RC_16M 时钟作为 timer 的驱动源; 5: 使用 STD_12M 时钟作为 timer 的驱动源; 6: 使用 STD_24M 时钟作为 timer 的驱动源; 7: 使用 STD_48M 时钟作为 timer 的驱动源; 15: 使用 IO mux in 时钟作为 timer 的驱动源; (crossbar)
9	PWM_INV	rw	PWM_INV: PWM 信号输出反向。
8	PWM_EN	rw	PWM_EN: PWM 信号输出使能。此位置 1 后, 相应 IO 口的功能将会被 PWM 信号输出替代。
7-4	PSET[3:0]	rw	PSET3-0: 预分频选择位 0000: 预分频 1 0001: 预分频 4 0010: 预分频 16 0011: 预分频 64 0100: 预分频 1*2 0101: 预分频 4*2 0110: 预分频 16*2 0111: 预分频 64*2 1000: 预分频 1*256 1001: 预分频 4*256 1010: 预分频 16*256 1011: 预分频 64*256 1100: 预分频 1*2*256 1101: 预分频 4*2*256

			1110: 预分频 16*2*256 1111: 预分频 64*2*256
3-2	CSEL[1:0]	rw	捕获模式端口选择: 0: IO mux in (crossbar) 1: IRFLT_OUT
1-0	MODE[1:0]	rw	MODE1-0: 工作模式选择 00: timer 关闭; 01: 定时/计数模式; 10: IO 口上升沿捕获模式 (当 IO 上升沿到来时, 把 TxCNT 的值捕捉到 TxPR 中); 11: IO 口下降沿捕获模式 (当 IO 下降沿到来时, 把 TxCNT 的值捕捉到 TxPR 中)。

2.Tx_CNT: timer x counter register

Bit	Name	RW	Description
31-0	Tx_CNT	rw	timer32 的计数寄存器

3.Tx_PR: timer x period register

Bit	Name	RW	Description
31-0	Tx_PR	rw	timer32 的周期寄存器

在定时/计数模式下, 当 Tx_CNT == Tx_PR 时, Tx_CNT 会被清 0。

在上升沿/下降沿捕获模式下, TxPR 是作为捕获寄存器使用的, 当捕获发生时, Tx_CNT 的值会被复制到 Tx_PR 中。而此时 Tx_CNT 自由的由 0-4294967295-0 计数, 不会和 Tx_PR 进行比较清 0。

4.Tx_PWM: timer x PWM register

Bit	Name	RW	Description
-----	------	----	-------------

31-0	Tx_PWM	rw	Timer32 的 PWM 设置寄存器
------	--------	----	---------------------

在 PWM 模式下，此寄存器的值决定 PWM 输出的占空比。占空比 N 的计算公式如下：

$$N = (Tx_PWM / Tx_PR) * 100\%$$

此寄存器不带有缓冲，写此寄存器的动作将可能导致不同步状态产生的 PWM 波形占空比瞬间过大或过小的问题。

珠海市杰理科技股份有限公司

第 4 章. ADC

4.1. 概述

10Bit ADC(A/D 转换器)，其时钟最大不可超过 1MHz。

4.2. 寄存器说明

1.ADC_CON: ADC control register

Bit	Name	RW	Description
31-16	-	-	预留
15-12	WAIT_TIME	rw	WAIT_TIME: 启动延时控制，实际启动延时为此数值乘 8 个 ADC 时钟
11-8	CH_SEL	rw	CH_SEL: 通道选择 0000: 选择 PA3 0001: 选择 PA5 0010: 选择 PA6 0011: 选择 PC3 0100: 选择 PC4 0101: 选择 PC5 0110: 选择 PB1 0111: 选择 PB2 1000: 选择 PB7 1001: 选择 PB6 1010: 选择 USBDP 1011: 选择 USBDM 1100: 选择 PB4 1101: 选择 PMU_DET_dnt 1110: 选择 AUDIO_DET_dnt

			1111: 选择 BT_DET_dnt(PLL,PMU,AUDIO,BT,FM 注意同一时间只能开一个通道, AUDIO 部分 en 是写 1 才是关闭)
7	PND	r	PND: 只读, 中断请求位, 当 ADC 完成一次转换后, 此位会被置为‘1’, 需由软件清‘0’
6	CPND	w	CPND: 只写, 写‘1’清除中断请求位, 写‘0’无效
5	ADC_IE	rw	ADC_IE: ADC 中断允许
4	ADC_EN	rw	ADC_EN: ADC 控制器使能
3	ADC_AE	rw	ADC_AE: ADC 模拟模块使能
2-0	ADC_BAUD	rw	ADC_BAUD: ADC 时钟频率选择 000: LSB 时钟 1 分频 001: LSB 时钟 6 分频 010: LSB 时钟 12 分频 011: LSB 时钟 24 分频 100: LSB 时钟 48 分频 101: LSB 时钟 72 分频 110: LSB 时钟 96 分频 111: LSB 时钟 128 分频

2.P3_ANA_CON4 ANA control SFR (PMU to ADC)

Bit	Name	RW	Description
7-5	reserved	rw	预留
4	VGB_SEL	rw	0: main VBG 1: weak VBG
3-1	CHANNEL_ADC_S[3:0]	rw	CHANNEL_ADC_S3-0:select channel to ADC 0000(default): VBG08 0001: VDC12 0010: SYSVDD

			0011: VTEMP 0100: PROGf 0101: 1/4 VBAT 0110: 1/4 LDO5v 0111: TWVD 1000: PVDD 1001: RVDD 1010: VSW 1011: PROGI 1100: EVDD 1101: VBGW08
0	PMU_DET_EN	r	PMU_DET_EN:PMU voltage detect to ADC output enable

3.DAA_CON0: DAA control register (AUDIO to ADC)

Bit	Name	RW	Description
6	VOUTR_TEST_EN_10v	rw	0: VOUTR to SARADC (0 是开启, 1 是关闭)
5	VOUTL_TEST_EN_10v	rw	0: VOUTL to SARADC (0 是开启, 1 是关闭)
4	QT_VREF_TEST_10v	rw	QT_VREF to SARSDC (0 是开启, 1 是关闭)
3	QT_AVDD11_TEST_10v	rw	QT_AVDD11 to SARSDC (0 是开启, 1 是关闭)
2	MICLDO_TEST_10v	rw	1: MICLDO to SARADC
1	DACVDD_TEST_EN_10v	rw	0: DACVDD to SARADC (0 是开启, 1 是关闭)
0	CTADCREf_TEST_10v	rw	1: CTADCREf to SARADC

【注意】： only one TEST channel can set to 1 while others must set to 0.

4.PLL_CON1: pll to adc

Bit	Name	RW	Description
18	PLL_TEST_EN	rw	pll to adc enable

17-16	PLL_TEST_S<1:0>	rw	00bias current 10u (CP 不测) -- 01PLL analog block voltage0.9v~1.2v 10PLL digital block voltage0.9v~1.2v Pl output 192meg clock (环路测试实现)
-------	-----------------	----	---

5.WLA_CON0 : BT to adc

Bit	Name	RW	Description
11-14	BT_TEST_S[3:0]	rw	see follow table
10	BT_TEST_EN_10v	rw	BT test en 0: off 1: open
8-9	BTCP_TS[1:0]	rw	BT to ADC Vout sel 00: 高阻 01: AOUTA 10: AOUTB 11: AOUTA 与 AOUTB 冲突, 不建议打开

6.BT_TEST_S[3:0]

BT_TEST_S3	BT_TEST_S2	BT_TEST_S1	BT_TEST_S0	AOUTA	AOUTB
0	0	0	0	DMIXER_I_diff	DMIXER_Q_diff
0	0	0	1	LPF_I_diff	LPF_Q_diff
0	0	1	0	Dmixer_I_P	Dmixer_I_N
0	0	1	1	Dmixer_Q_P	Dmixer_q_N
0	1	0	0	LPF_I_P	LPF_I_N
0	1	0	1	LPF_Q_P	LPF_Q_N
0	1	1	0	TX_TESTIP	TX_TESTQP
0	1	1	1	TX_TESTIN	TX_TESTQN

1	0	0	0	RX_TOUT	
---	---	---	---	---------	--

珠海市杰理科技股份有限公司

第 5 章. IIC

5.1. 概述

IIC 接口是一个标准的遵守 IIC 协议的串行通讯接口。在上面传输的数据以 Byte（8bit）为最小单位，且永远是 MSB 在前。

IIC 接口支持主机和从机两种模式

主机：

1) IIC 接口时钟由本机产生，提供给片外 IIC 设备使用；

2) IIC 接口的驱动时钟可配置，频率范围为 $F = F_{\text{system}} / (\text{IIC_BAUD} * 2)$ + 电阻上拉的时间。

上拉电阻越大，频率越低。

从机：

1) IIC 接口时钟由片外 IIC 设备产生，提供给本机使用；

2) IIC 总线上每一个 start/restart 位后接从机地址，此时当外部 IIC 设备所发的地址与我们匹配时，硬件会自动回应（ack 位上为“0”）；

IIC 接口使用下降沿更新数据，上升沿采集数据。

IIC 接口的发送寄存器和接收寄存器在物理上是分开的，但在逻辑上它们一起称为 IIC_BUF 寄存器，使用相同的 SFR 地址。当写这个 SFR 地址时，写入至发送寄存器。当读这个 SFR 地址时，从接收寄存器读出。

当作为 Master 的传送数据，可以在发送完 1byte 的数据之后，待 pnding 起来后，通过往 buf 中写数据再次传输数据，也可以配置 CON0[5]来结束本次传输。

如果是作为 Master 接收数据，那么传送完地址后，接收到第一笔数据，就自动结束本次传输。

寄存器基地址为（0x24）

5.2. 寄存器说明

1.IIC_CON0: IIC control register0

Bit	Name	RW	Description
-----	------	----	-------------

31	iic_pnding	r	PND: 中断请求标志, 当完成 1Byte 加 1 应答位的传输后会被硬件置“1”; 清除此标志用软件方式: 写 bit[7]。
30	iic_pnding_ab	r	当作为主机时, 发送的地址, 没有从机应答的 pnding, 或者, 发送的数据没有应答的 pnding, 写 bit[7]清除该 pnding
29	iic_stop_pnding	r	当 IIC 停止时, pnding 标志位 (不产生中断)
28	slv_is_t	r	master 要求从机是发送的标志位 1: 从机是发送 0: 从机是接收
27	slv_t_done	r	当作为 slave, 是发送时, 数据结束的标志, 会在 master 下次发送 start 时, 自动清 0 1: 数据传输结束 (master 回应了一个 NACK) 0: 数据还要继续传输 (master 回应了一个 ACK)
26-17	reserved	rw	reserved
17	slv_dma_small_dian	rw	作为 slave, 且接收的时候, dma 打开时, 小端数据输出使能 1: 小端模式 0: 大端模式
16	mst_rd_start	w	作为主机, 接收时, 触发下一笔读操作
15	slv_dma_en	rw	当作为 slave, 且接收时, dma 的使能 0: dma 关闭 1: dma 打开 (使能打开后, 在写满 buf 后, 才起 pnding, 而且在写满 buf 后, 如果主机继续发数据, SLV 会根据 BIT[13]来决定是否拉住时钟。在 DMA 期间, master 不能 restart, 不然 slv 会把 restart 后的地址当做数据处理)
14	iic_auto_stop_en	rw	IIC 硬件自动 stop 使能 1: 硬件检测到 NACK, 自动进入 stop 0: 硬件检测到 NACK, 会拉低 SCL, 等待 (是否进入 restart, 或者继续发送/读取数据, 或者进入 stop 的状态)

13	iic_restart	w	iic 重新开始位。（在 iic_pnding 产生之后，配置该 iic_restart，然后再配置 iic_buf，写入新的控制 byte 后，硬件会自动 restart 并发送新的控制 byte，发送完成后，产生 iic_pnding）
12	slv_clk_en	rw	作为 SLV 时，在传输完一个 btye 的数据（不包含地址）之后，是否要拉住时钟 0: 不拉 1: 拉住时钟（将 SCL 拉为 0）
11	iic_conf_ack	rw	iic 可配置 ack（最好一直设为 0，由硬件自动回复） 0: 硬件自动回复 ack/no_ack 1: 硬件强制回复 no_ack
10	iic_end_rd	w	当为主机接收，且为连续 byte 接收模式时，写“1”表示完成下一个 byte 数据交易后，stop 数据接收（须在开启最后一笔交易之后配置）
9	iic_rd_sel	rw	当为主机接收时 0: 单 byte 接收模式 1: 连续 byte 接收模式
8	iic_int_en	rw	IIC 中断使能。 0: 禁止 iic 中断 1: 允许中断允许
7	iic_pnding_clr	w	IIC 的 pnding 清除
6	iic_slv_continue	w	当作为从机时，完成 1byte 的数据传送时（不包括地址的校验），会将 SCL 拉低（配合 bit[12]使用），该 bit 写“1”会将 SCL 释放，继续交易数据。
5	iic_end_wr	w	IIC 为主机时，且选择发送时，写“1”停止发送（须在 pnding 起来之后配置）
4	iic_dat_dir	rw	当前的发送和接收选择 0: 发送 1: 接收

3	iic_isel	rw	iic_cki 和 iic_di 输入选择: 0: 选择 IO 直接输入 1: 选择 IO 经过 filter 后再输入
2	iic_slave	rw	IIC 接口模式选择 0: 主机模式 1: 从机模式
1	iic_start	w	IIC 开始准备工作
0	iic_en	rw	IIC 接口使能。 0: 关闭 IIC 接口 1: 打开 IIC 接口

2.IIC_BUF: IIC control register1

Bit	Name	RW	Description
7-0	IIC_BUF	rw	

发送寄存器和接收寄存器共用此 SFR 地址。写入至发送寄存器，从接收寄存器读出。

3.IIC_BAUD: IIC control register2

Bit	Name	RW	Description
15-0	IIC_BAUD	rw	

- 1) IIC 接口为主机时，IIC_BAUD 做为时钟设置寄存器。
- 2) 频率范围为 $F = F_{\text{system}} / (\text{IIC_BAUD} * 2) + \text{电阻上拉的时间}$ 。上拉电阻越大，频率越低。
- 3) IIC 接口为从机时，IIC_BAUD 做为从机地址(7bits)设置寄存器。

从机地址 = IIC_BAUD[7: 1]

当地址匹配时硬件自动应答 (ACK 位为 “0”)

4.IIC_CON1: IIC control register3

Bit	Name	RW	Description
31-0	IIC_CON1	rw	IIC 作为 SLAVE，接收时，dma 输出的起始地址

5.IIC_CON2: IIC control register4

Bit	Name	RW	Description
15-0	IIC_CNT	r	IIC 作为 SLAVE，接收时，已经写入了多少个 32bit 的数据
15-0	IIC_BUF_DEPTH	rw	IIC 作为 SLAVE，接收时，开辟的 buf 深度，（以 32bit 为单位）

程序示例如下：

作为 Master 和 transmitter:

```

SFR(JL_IIC->CON0, 2, 1, 0); //iic_mst
SFR(JL_IIC->CON0, 3, 1, 1); //iic_isel
SFR(JL_IIC->CON0, 4, 1, 0); //0:transmitter 1:receiver
SFR(JL_IIC->CON0, 8, 1, 1); //iic_int_en
SFR(JL_IIC->CON0, 9, 1, 0); //0:1byte_rd 1:continuous_rd

SFR(JL_IIC->BAUD, 0, 16, 20); //baud
SFR(JL_IIC->BUF, 0, 8, 4); //initial_adr and r/w
SFR(JL_IIC->CON0, 0, 1, 1); //iic_en

SFR(JL_IIC->CON0, 1, 1, 1); //iic_start

for(int i=0;i<10;i++) {
    while(!((JL_IIC->CON0)>>31)); //wait pnding
    SFR(JL_IIC->CON0, 7, 1, 1); //iic_pnding_clr
    asm("csync");
    SFR(JL_IIC->BUF, 0, 8, i+5); //initial_adr and r/w
}

```

```
}
```

```
while(!((JL_IIC->CON0)>>31)); //wait pnding  
SFR(JL_IIC->CON0, 7, 1, 1); //iic_pnding_clr  
SFR(JL_IIC->CON0, 5, 1, 1); //iic_wr_end  
delay(500);
```

作为 slv 和 receiver:

```
SFR(JL_IIC->CON0, 2, 1, 1); //0:mst 1:slave  
SFR(JL_IIC->CON0, 3, 1, 1); //iic_isel  
SFR(JL_IIC->CON0, 4, 1, 1); //0:transmitter 1:receiver  
SFR(JL_IIC->CON0, 8, 1, 1); //iic_int_en  
  
SFR(JL_IIC->BAUD, 0, 16, 1); //baud  
SFR(JL_IIC->CON0, 0, 1, 1); //iic_en  
SFR(JL_IIC->CON0, 12, 1, 0); //0:slv no stretch clk 1:slv stretch clk  
  
SFR(JL_IIC->CON0, 1, 1, 1); //iic_start
```

作为 Mst 和 receiver, 而且连续读:

```
SFR(JL_IIC->CON0, 2, 1, 0); //iic_mst  
SFR(JL_IIC->CON0, 3, 1, 1); //iic_isel  
SFR(JL_IIC->CON0, 4, 1, 1); //0:transmitter 1:receiver  
SFR(JL_IIC->CON0, 8, 1, 1); //iic_int_en  
SFR(JL_IIC->CON0, 9, 1, 1); //0:1byte_rd 1:continuous_rd  
  
SFR(JL_IIC->BAUD, 0, 16, 20); //baud  
SFR(JL_IIC->BUF, 0, 8, 4); //initial_adr and r/w
```



```
SFR(JL_IIC->CON0, 0, 1, 1); //iic_en
```

```
SFR(JL_IIC->CON0, 1, 1, 1); //iic_start
```

```
for(int i=0;i<10;i++) {
```

```
while(!((JL_IIC->CON0)>>31)); //wait pnding
```

```
SFR(JL_IIC->CON0, 7, 1, 1); //iic_pnding_clr
```

```
asm("csync");
```

```
int tmp = JL_IIC->BUF;
```

```
}
```

```
while(!((JL_IIC->CON0)>>31)); //wait pnding
```

```
SFR(JL_IIC->CON0, 10, 1, 1); //iic_end_rd
```

```
SFR(JL_IIC->CON0, 7, 1, 1); //iic_pnding_clr
```

```
int tmp = JL_IIC->BUF; //1st rd
```

```
delay(500);
```

作为 Mst 和 receiver, 读 1byte:

```
SFR(JL_IIC->CON0, 2, 1, 0); //iic_mst
```

```
SFR(JL_IIC->CON0, 3, 1, 1); //iic_isel
```

```
SFR(JL_IIC->CON0, 4, 1, 1); //0:transmitter 1:receiver
```

```
SFR(JL_IIC->CON0, 8, 1, 1); //iic_int_en
```

```
SFR(JL_IIC->CON0, 9, 1, 0); //0:1byte_rd 1:continuous_rd
```

```
SFR(JL_IIC->BAUD, 0, 16, 20); //baud
```

```
SFR(JL_IIC->BUF, 0, 8, 4); //initial_adr and r/w
```

```
SFR(JL_IIC->CON0, 0, 1, 1); //iic_en
```

```
SFR(JL_IIC->CON0, 1, 1, 1); //iic_start
```

```
while(!((JL_IIC->CON0)>>31)); //wait pnding
```

```
int tmp = JL_IIC->BUF; //1st rd
```

```
SFR(JL_IIC->CON0, 7, 1, 1); //iic_pnding_clr
```

```
delay(500);
```

作为 slave 和 transmitter:

```
SFR(JL_IIC->CON0, 2, 1, 1); //iic_mst
```

```
SFR(JL_IIC->CON0, 3, 1, 1); //iic_isel
```

```
SFR(JL_IIC->CON0, 4, 1, 0); //0:transmitter 1:receiver
```

```
SFR(JL_IIC->CON0, 8, 1, 1); //iic_int_en
```

```
SFR(JL_IIC->BAUD, 0, 16, 2); //baud
```

```
SFR(JL_IIC->CON1, 0, 8, 85); //wr_buf
```

```
SFR(JL_IIC->CON0, 0, 1, 1); //iic_en
```

```
SFR(JL_IIC->CON0, 12, 1, 0); //0:slv no stretch clk 1:slv stretch clk
```

```
SFR(JL_IIC->CON0, 1, 1, 1); //iic_start
```

第6章. 红外过滤 (IRFLT)

6.1. 概述

IRFLT 是一个专用的硬件模块，用于去除掉红外接收头信号上的窄脉冲信号，提升红外接收解码的质量。IRFLT 使用一个固定的时基对红外信号进行采样，必须连续 4 次采样均为‘1’时，输出信号才会变为‘1’；必须连续 4 次采样均为‘0’时，输出信号才会变为‘0’。换言之，脉宽小于 3 倍时基的窄脉冲将被滤除。改变该时基的产生可兼容不同的系统工作状态，也可在一定范围内调整对红外信号的过滤效果。通过对 IOMC (IO re-mapping) 寄存器的配置，可以将 IRFLT 插入到系统 6 个 timer 中某一个的捕获引脚之前。

例如通过 IOMC 寄存器选择了 IRFLT 对 timer1 有效，并且 IRFLT_EN 被使能之后，则 IO 口的信号会先经过 IRFLT 进行滤波，然后再送至 timer1 中进行边沿捕获。

1. 硬件接口

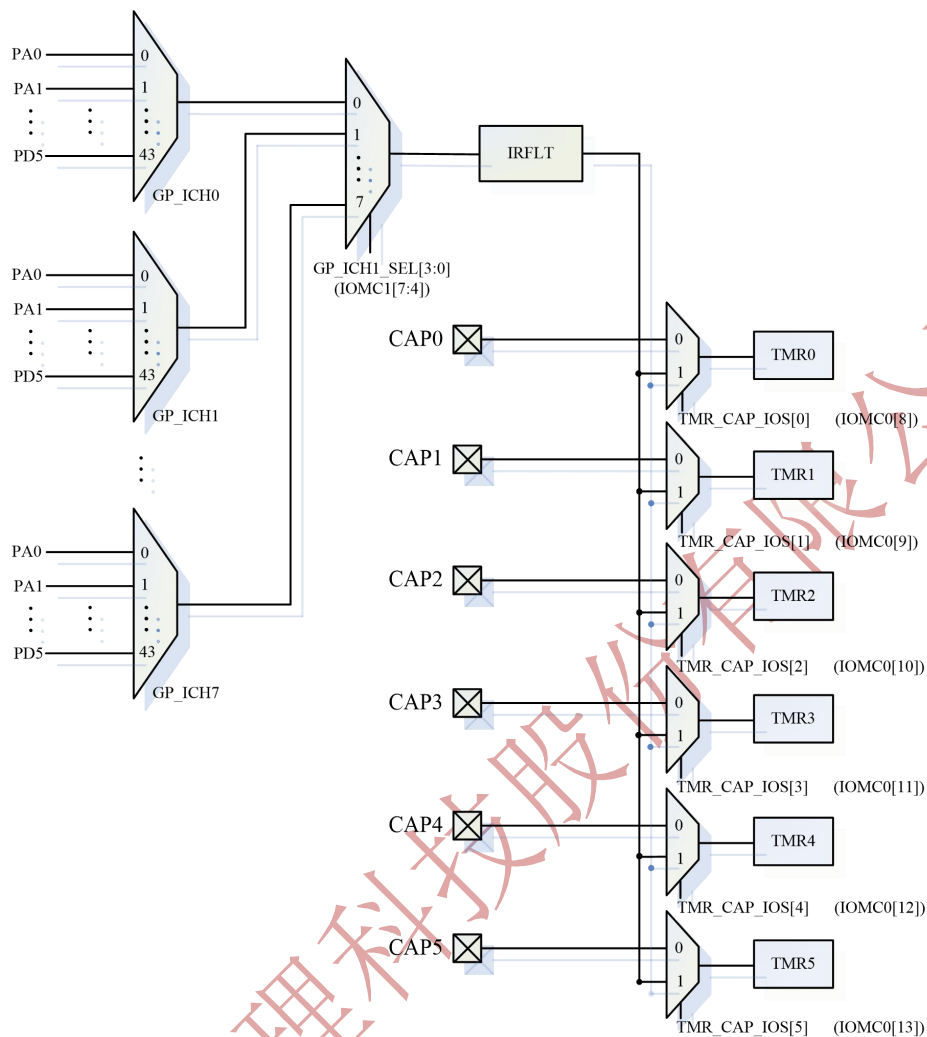


图 6-1 硬件接口示意图

2. 时基选择

PSEL 选定的分频倍数 N 和 TSRC 选定的驱动时钟的周期 T_c 共同决定了 IRFLT 用于采样红外接收信号的时基 T_s :

$$T_s = T_c * N$$

例如，当选择 32KHz 的 OSC 时钟，并且分频倍数为 1 时， $T_s = 30.5\mu s$ 。根据 IRFLT 的工作规则，所有小于 $(30.5 * 3 = 91.5\mu s)$ 的窄脉冲信号，均会被滤除。

又如，当选择 48MHz 的系统时钟，并且分频倍数为 1024 时， $T_s = 21.3\mu s$ 。根据 IRFLT 的工作规则，所有小于 $(21.3 * 3 = 63.9\mu s)$ 的窄脉冲信号，均会被滤除。

6.2. 寄存器说明

1. IRFLT_CON: irda filter contrl register (8bit addressing)

Bit	Name	RW	Description
7-4	PSEL[3:0]	rw	PSEL[3-0]: 时基发生器分频选择 0000: 分频倍数为 1; 0001: 分频倍数为 2; 0010: 分频倍数为 4; 0011: 分频倍数为 8; 0100: 分频倍数为 16; 0101: 分频倍数为 32; 0110: 分频倍数为 64; 0111: 分频倍数为 128; 1000: 分频倍数为 256; 1001: 分频倍数为 512; 1010: 分频倍数为 1024; 1011: 分频倍数为 2048; 1100: 分频倍数为 4096; 1101: 分频倍数为 8192; 1110: 分频倍数为 16384; 1111: 分频倍数为 32768;
3-2	TSRC[1-0]	r	TSRC[1-0]: 时基发生器驱动源选择 00: 选择 LSB_CLK 来驱动时基发生器; 01: 选择 RC 时钟来驱动时基发生器; 10: 选择 OSC_CLK 时钟来驱动时基发生器; 11: 选择 PLL_48M 时钟来驱动时基发生器;
1	reserved	r	预留

0	IRFLT_EN	rw	IRFLT_EN: IRFLT 使能 0: 关闭 IRFLT; 1: 打开 IRFLT;
---	----------	----	--

6.3. 例程

```
#define IR_PA8      0//PA8

void IR_init(void)
{
    ///选择 IO

    PORTA_DIR |= BIT(8);

    IOMC0 &= ~(0xF<<8);
    IOMC0 |= (IR_CTR<<8);//选择红外输入脚
    IOMC0 |= (2<<4);    //选择捕获 timer

    if(get_apb_clk() >= 12000000L)
    {
        IRFLT_CON = 0xa1;    //1024 分频倍数, 使能 IR
        TMR2_CON  = 0x33;    //预分频 64, IO 口下降沿捕获
    }
    else
    {
        IRFLT_CON = 0x41;    //4096 分频倍数, 使能 IR
        TMR2_CON  = 0x23;    //预分频 64, IO 口下降沿捕获
    }
}
```

```
timer2_pad = (get_apb_clk()/1000)/64;
```

```
printf("timer 2 init\n");  
  
int_enter_pr[TIMER2_INT-1] = timer2_isr;  
  
}
```

珠海市杰理科技股份有限公司

第 7 章. PULSE COUNTER

7.1. 概述

以下为 pulse_counter/触摸键的寄存器说明及使用方法。

7.2. 寄存器说明

1.PL_CNT_CON

Bit	Name	RW	Description
7-4	-	-	预留
3-2	clk_sel	rw	pulse counter 时钟选择 00: 选择 osc_clk 作为计数时钟; 01: 选择 clk_mux_in 作为计数时钟 (iomc1[15:12] input channel3 选择); 10: 选择 pll_192m 作为计数时钟; 11: 选择 pll_240m 作为计数时钟; 备注: 时钟选择的频率越大, pl_cnt_value 计数值会越大, 分辨率会越高, 触摸键的灵敏度也越高
1	en	rw	触摸键使能, 当 cap_mux_in (input channel 2) 为 1 时, PLCNTVL 累加计数, 计满后归 0 再重新累加 0: 触摸键禁止; 1: 触摸键使能
0	test_en	rw	触摸键测试使能 (此位专用于测试) 0: 测试模式未使能; 1: 测试模式使能;

【注意】：当 cap_mux_in (input channel 2) 选择 TMRx_PWMOUT, pulse counter 时钟选择 clk_mux_in (input channel 3) 时, 可以用内部固定周期的 PWM, 计算芯片 IO 上不确定的时钟频率。

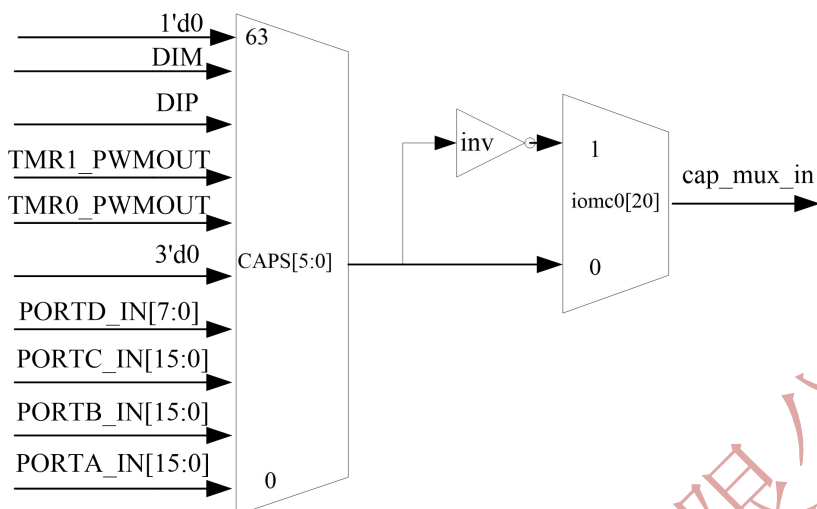


图 8-1 cap_mux_in(input channel 2)电路控制图

7.3. 示例代码

以下以 PA1 为例:

1. 变量定义:

```
int Touchkey_value_old, Touchkey_value_new, Touchkey_value_delta;    //32bit 有符号整数
```

2. 选择检测管脚

```
IOMC2 &= ~(0xff00);    //
IOMC2 |= 0x01 << 16;    //选择 PA1

PORTA_OUT |= BIT(1);
PORTA_DIR &= ~BIT(1);    //PA1 输出为 1
PORTA_PD |= BIT(1);    //PA1 下拉打开
```

3. 初始化计数器配置

```
PLCNTCON &= ~(0xC);    //
PLCNTCON |= BIT(3);    //选择 pll_192m 作为触摸键时钟
PLCNTCON |= BIT(1);    //使能计数器
```

4.检测电容

```
Touchkey_value_old = PLCNTVL; //读取旧值  
PORTA_DIR |= BIT(1); //对应管脚输入使能  
While(PORTA_IN & BIT(1));  
Touchkey_value_new = PLCNTVL;  
If(Touchkey_value_old > Touchkey_value_new)then  
Touchkey_value_new += 0x10000;  
Touchkey_value_delta = Touchkey_value_new - Touchkey_value_old;
```

第 8 章. RDEC

8.1. 概述

RDEC (rotate decoder) 是一个用于旋转编码器检测的模块，它支持两线输入的旋转编码器，可以检测旋转方向和旋转步数。

本芯片中，支持 1 路 RDEC，中断号为 25。

表 23-1 各 RDEC 使用的 IO 端口列表

模块	信号名称	IO 选择(default)
RDEC0	RDEC0_SIN[0]	PB6
	RDEC0_SIN[1]	PB7

【注意】：以上是 RDEC 在该项目必须支持的 IO 映射，当然，也可以通过 Crossbar 映射其他普通 IO 使用。

表 23-2 每路 RDEC 使用的 SFR 列表

SFR 名称	描述
RDEC_CON	RDEC 控制寄存器
RDEC_DAT	RDEC 结果读出寄存器

8.2. 控制寄存器

1. RDEC_CON 8bit 宽度

Bit	Name	RW	Description
7	PND	r	PND, 只读。写入无效。
6	CPND	w	CPND, 只写。写入 1 清除 PND, 读出永远为 0。
5-2	RDEC_SPD	rw	RDEC_SPD, 采样速率设置

			$Tsr = (2^{\wedge} RDEC_SPD) / Flsb$ Flsb 为低速外设总线的频率, 软件应当设置 RDEC_SPD 使 Tsr 介于 0.5~2mS 之间
1	RDEC_POL	rw	RDEC_POL 0: 输入引脚无信号时处于 1 状态 (外部引脚上拉) 1: 输入引脚无信号时处于 0 状态 (外部引脚下拉)
0	EN	rw	RDEC_EN 0: 模块关闭 1: 模块打开

2.RDEC_DAT 8bit 宽度, 只读

Bit	Name	RW	Description
7-0	RDEC_DAT	r	此寄存器为 8 位有符号数, 表示旋转编码器正反向旋转的步数。 当检测到旋转编码器动作时, PND 会设置为 1, 软件可通过中断或查询方式来读取此寄存器。也可以完全不理睬 PND, 软件隔一段时间来访问此寄存器, 但需注意检测时间不能过长, 如果此寄存器的数值超过-128~127 的区间, 将会产生溢出, 无法表示出旋转编码器的正确动作。 【注意】: 清 PND 的动作会将此寄存器一同清 0。

第 9 章. SPI

9.1. 概述

SPI 接口是一个标准的遵守 SPI 协议的串行通讯接口，在上面传输的数据以 Byte（8bit）为最小单位，且永远是 MSB 在前。

【注意】：该模块所有说明及配置仅作为 SPIx 模块使用，不能套用到 SFC 中

SPI 接口支持主机和从机两种模式

主机：SPI 接口时钟由本机产生，提供给片外 SPI 设备使用

从机：SPI 接口时钟由片外 SPI 设备产生，提供给本机使用

工作于主机模式时，SPI 接口的驱动时钟可配置，范围为 系统时钟~系统时钟/256

工作于从机模式时，SPI 接口的驱动时钟频率无特殊要求，但数据速率需要进行限制，否则易出现接收缓冲覆盖错误。

SPI 接口可独立地选择在 SPI 时钟的上升沿或下降沿更新数据，在 SPI 时钟的上升沿或下降沿采样数据。

SPI 接口支持单向（Unidirection）和双向（Bidirection）模式

单向模式：使用 SPICK 和 SPIDAT 两组连线，其中 SPIDAT 为双向信号线，同一时刻数据只能单方向传输。

双向模式：使用 SPICK，SPIDI 和 SPIDO 三组连线，同一时刻数据双向传输。但 DMA 不支持双向数据传输，当在本模式下使能 DMA 时，也只有一个方向的数据能通过 DMA 和系统进行传输。

SPI 单向模式支持 1bit data、2bit data 和 4bit data 模式，即：

1bit data 模式：串行数据通过一根 DAT 线传输，一个字节数据需 8 个 SPI 时钟。

2bit data 模式：串行数据通过两根 DAT 线传输，一个字节数据需 4 个 SPI 时钟。

4bit data 模式：串行数据通过四根 DAT 线传输，一个字节数据需 2 个 SPI 时钟。

【注意】：SPI0 支持 1bit, 2bit 和 4bit 模式，SPI1/SPI2 只支持 1bit/2bit 模式

SPI 双向模式只支持 1bit data 模式，即：

1bit data 模式：串行数据通过一根 DAT 线传输，一个字节数据需 8 个 SPI 时钟。

SPI 接口在发送方向上为单缓冲，在上一次传输未完成之前，不可开始下一次传输。在接收方向上为双缓冲，如果在下一次传输完成时 CPU 还未取走本次的接收数据，那么本次的接收数据将会丢失。

SPI 接口的发送寄存器和接收寄存器在物理上是分开的，但在逻辑上它们一起称为 SPIBUF 寄存器，使用相同的 SFR 地址。当写这个 SFR 地址时，写入至发送寄存器。当读这个 SFR 地址时，从接收寄存器读出。

SPI 传输支持由 CPU 直接驱动，写 SPIBUF 的动作将启动一次 Byte 传输。

SPI 传输也支持 DMA 操作，但 DMA 操作永远是单方向的，即一次 DMA 要么是发送一包数据，要么是接收一包数据，不能同时发送并且接收一包数据，即使在双向模式下也是这样。每次 DMA 操作支持的数据量为 1-65535Byte。写 SPIADR 的动作将启动一次 DMA 传输。

9.2. 控制寄存器

寄存器列表	SPI0	SPI1	SPI2
SPIx_CON	SPI0_CON	SPI1_CON	SPI2_CON
SPIx_BUF	SPI0_BUF	SPI1_BUF	SPI2_BUF
SPIx_BAUD	SPI0_BAUD	SPI1_BAUD	SPI2_BAUD
SPIx_ADR	SPI0_ADR	SPI1_ADR	SPI2_ADR
SPIx_CNT	SPI0_CNT	SPI1_CNT	SPI2_CNT

1.SPIx_CON: SPIx control register (16bit addressing)

Bit	Name	RW	Description
15	PND	r	中断请求标志，当 1Byte 传输完成或 DMA 传输完成时会被硬件置 1。 有 3 种方法清除此标志 向 PCLR 写入‘1’ 写 SPIBUF 寄存器来启动一次传输 写 SPICNT 寄存器来启动一次 DMA
14	PCLR	w	软件在此位写入‘1’将清除 PND 中断请求标志。
13	IE	rw	SPI 中断使能 0: 禁止 SPI 中断 1: 允许中断允许
12	DIR	rw	在单向模式或 DMA 操作时设置传输的方向 0: 发送数据 1: 接收数据
11-10	DATW	r	SPI 数据宽度设置 00: 1bit 数据宽度 01: 2bit 数据宽度 10: 4bit 数据宽度 11: NA，不可设置为此项 (注: SPI0 支持 1bit, 2bit 和 4bit 模式, SPI1/SPI2 只支持 1bit 模式)
9	reserved	r	0
8	reserved	r	0
7	CSID	rw	SPICS 信号极性选择 0: SPICS 空闲时为 0 电平 1: SPICS 空闲时为 1 电平

6	CKID	rw	SPICK 信号极性选择 0: SPICK 空闲时为 0 电平 1: SPICK 空闲时为 1 电平
5	UE	rw	更新数据边沿选择 0: 在 SPICK 的上升沿更新数据 1: 在 SPICK 的下降沿更新数据
4	SE	rw	采样数据边沿选择 0: 在 SPICK 的上升沿采样数据 1: 在 SPICK 的下降沿采样数据
3	BIDIR	rw	单向/双向模式选择 0: 单向模式，数据单向传输，同一时刻只能发送或者接收数据。 数据传输方向因收发而改变，所以由硬件控制，不受写 IO 口 DIR 影响。 1: 双向模式，数据双向传输，同时收发数据，但 DMA 只支持一个方向的数据传输。 数据传输方向设置后不改变，所以由软件控制，通过写 IO 口 DIR 控制。
2	CSE	rw	SPICS 信号使能 0: 不使用 SPICS 信号 1: 使用 SPICS 信号
1	SLAVE	rw	从机模式
0	SPIE	rw	SPI 接口使能

			0: 关闭 SPI 接口
			1: 打开 SPI 接口

2.SPIx_BAUD: SPI baudrate setting register (8bit addressing, write only).

Bit	Name	RW	Description
7-0	SPI_BAUD	rw	

SPI 主机时钟设置寄存器

$SPICK = \text{system clock} / (\text{SPIBAUD} + 1)$

3.SPI_BUF: SPI buffer register (8bit addressing)

Bit	Name	RW	Description
7-0	SPI_BUF	rw	

发送寄存器和接收寄存器共用此 SFR 地址。写入至发送寄存器，从接收寄存器读出。

4.SPI_ADR: SPI DMA start address register (25bit addressing, write only).

Bit	Name	RW	Description
24-0	SPI_ADR	rw	

SPI DMA 起始地址寄存器，只写，读出为不确定值

5.SPI_CNT: SPI DMA counter register (16bit addressing, write only)

Bit	Name	RW	Description
15-0	SPI_CNT	rw	

SPI DMA 计数寄存器，只写，读出为不确定值

此寄存器用于设置 DMA 操作的数目（按 Byte 计）并启动 DMA 传输。

如，需启动一次 512Byte 的 DMA 传输，写入 0x0200，此写入动作将启动本次传输。

9.3. 传输波形

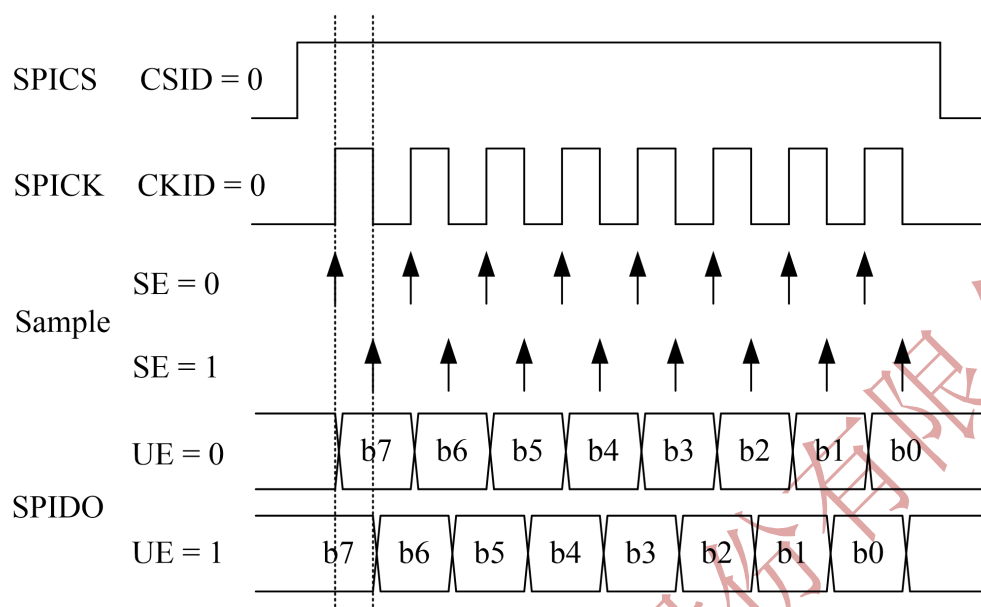


图 10-1 传输波形图 1

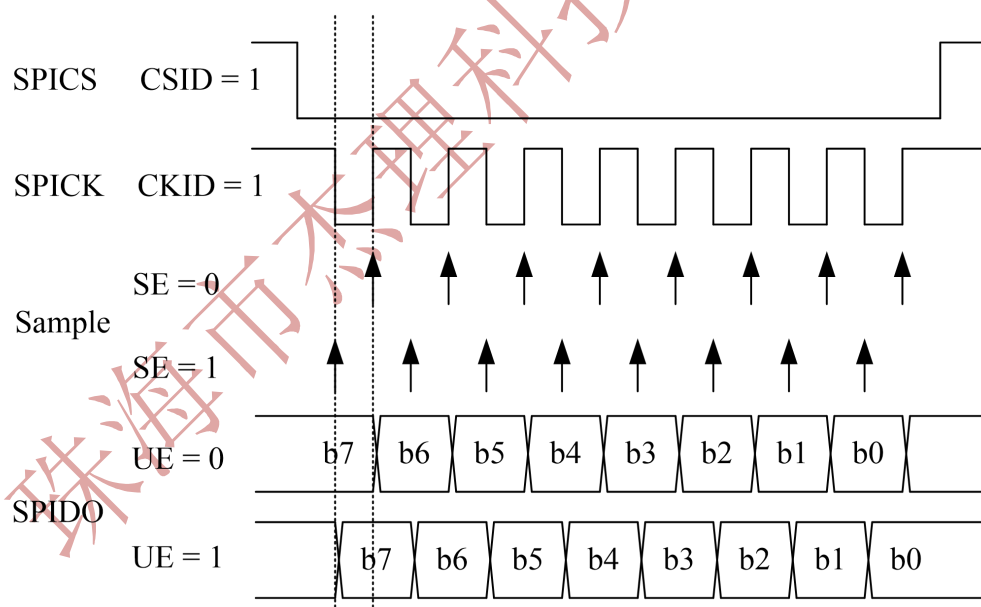


图 10-2 传输波形图 2

第 10 章. UART

10.1. 概述

UART0、 UART1 支持接收带循环 Buffer 的 DMA 模式和普通模式。

UART2 只支持普通模式，不支持 DMA 模式。

UART0 在 DMA 接收的时候有一个循环 Buffer，UTx_RXSADR 表示它的起始，UTx_RXEADR 表示它的结束。同时，在接收过程中，会有一个超时计数器（UTx_OTCNT），如果在指定的时间里没有收到任何数据，则超时中断就会产生。超时计数器是在收到数据的同时自动清空。

10.2. 控制寄存器

寄存器列表	UART0	UART1	UART2
UTx_CON(0)	UT0_CON0	UT1_CON	UT2_CON
UTx_CON1	UT0_CON1	UT1_CON1	UT2_CON1
UTx_BAUD	UT0_BAUD	UT1_BAUD	UT2_BAUD
UTx_BUF	UT0_BUF	UT1_BUF	UT2_BUF
UTx_TXADR	UT0_TXADR	UT1_TXADR	UT2_TXADR
UTx_TXCNT	UT0_TXCNT	UT1_TXCNT	UT2_TXCNT
UTx_RXCNT	UT0_RXCNT	UT1_RXCNT	UT2_RXCNT
UTx_RXSADR	UT0_RXSADR	UT1_RXSADR	UT2_RXSADR
UTx_RXEADR	UT0_RXEADR	UT1_RXEADR	UT2_RXEADR
UTx_HRXCNT	UT0_HRXCNT	UT1_HRXCNT	UT2_HRXCNT
UTx_OTCNT	UT0_OTCNT	UT1_OTCNT	UT2_OTCNT

1. UTx_CON(0): uart x control register (16bit addressing).

Bit	Name	RW	Description
-----	------	----	-------------

15	TPND	r	TPND:TX Pending
14	RPND	r	RPND:RX Pending& Dma_Wr_Buf_Empty, 数据接收不完 Pending 不会为 1
13	CLRTPND	r	CLRTPND: 清空 TX Pending
12	CLRRPND	r	CLRRPND: 清空 RX Pending
11	OTPND	r	(*) OTPND:OverTime Pending
10	CLR_OTPND	r	(*) CLR_OTPND: 清空 OTPND
9			
8			
7	RDC	w	RDC (*): 写 1 时, 将已经收到的数目写到 UTx_HRXCNT, 已收到的数目清零写 0 无效
6	RX_MODE	rw	RXMODE (*): 读模式选择 0:普通模式, 不用 DMA; 1:DMA 模式。
5	OT_IE	rw	OTIE (*):OT 中断允许 0: 不允许; 1:允许。
4	DIVS	rw	DIVS:前 3 分频选择, 0 为 4 分频, 1 为 3 分频
3	RXIE	rw	RXIE:RX 中断允许 当 RX Pending 为 1, 而且 RX 中断允许为 1, 则会产生中断
2	TXIE	rw	TXIE:TX 中断允许 当 TX Pending 为 1, 而且 TX 中断允许为 1, 则会产生中断
1	UTRXEN	rw	UTRXEN:UART 模块接收使能
0	UTTXEN	rw	UTTXEN:UART 模块发送使能

(*) : 只有 UART0/1 支持

2. UTx_CON1: uart x control register1 (16bit addressing).

Bit	Name	RW	Description
15	CTSPND	rw	CTSPND:CTS 中断 pending

14	CLR_CTSPND	wo	CLR_CTSPND: 清楚 CTS pending
13	CLRRTS	wo	CLRRTS: 清除 RTS 0:N/A 1:清空 RTS
12-5	reserved	rw	预留
4	RX_DISABLE	rw	关闭数据接收 0:开启输入（正常模式） 1:关闭输入（输入固定为 1）
3	CTSIE	rw	CTSIE:CTS 中断使能 0:禁止中断; 1:中断允许 【注意】 : 只有 UART1 有该功能
2	CTSE	rw	CTSE:CTS 使能 0:禁止 CTS 硬件流控制 1: 允许 CTS 硬件流控制 【注意】 : 只有 UART1 有该功能
1	RTS_DMAEN	rw	RTS_DMAEN:RTS 接收数据流控制使能 0: 禁止 1:允许 【注意】 : 只有 UART1 有该功能
0	RTSE	rw	RTSE:RTS 使能 0:禁止 RTS 硬件流控制 1:允许 RTS 硬件流控制 【注意】 : 只有 UART1 有该功能

3. UTx_CON2: uart x control register2 (16bit addressing).

Bit	Name	RW	Description
-----	------	----	-------------

15-3			
2	RB8	r	RB8:9Bit 模式时。RX 接收到的第 9 位
1	TB8	rw	TB8:9Bit 模式时，TX 发送的第 9 位
0	M9EN	rw	M9EN:9bit 模式使能

4. UTx_BAUD: uart x baudrate register (16bit addressing, Write Only).

Bit	Name	RW	Description
15-0	UTx_BAUD	wo	

uart 的 UTx_DIV 的整数部分

串口频率分频器因子(UTx_DIV)的整数部分

当 DIVS=0 时，

$$\text{Baudrate} = \text{Freq_sys} / ((\text{UTx_BAUD} + 1) * 4 + \text{BAUD_FRAC})$$

当 DIVS=1 时，

$$\text{Baudrate} = \text{Freq_sys} / ((\text{UTx_BAUD} + 1) * 3 + \text{BAUD_FRAC})$$

(Freq_sys 是 apb_clk, 指慢速设备总线的时钟，非系统时钟)

5. UTx_BUF: uart x data buffer register (8bit addressing).

Bit	Name	RW	Description
15-0	UT0BUF	wo	uart 的收发数据寄存器 写 UTx_BUF 可启动一次发送; 读 UTx_BUF 可获得已接收到的数据。

6. UTx_TXADR: uart x TX DMA address(25bit addressing, Write Only)

Bit	Name	RW	Description
24-0	UTx_TXADR	wo	DMA 发送数据的起始地址

7. UTx_TXCNT: uart x TX DMA count (32bit addressing, Write Only).

Bit	Name	RW	Description
31-0	UTx_TXADR	wo	写 UTx_TXCNT，控制器产生一次 DMA 的操作，同时清空中断，当 uart 需要发送的数据达到 UTx_TXCNT 的值，控制器会停止发送数据的操作，同时产生中断（UTx_CON[15]）。

8. UTx_RXCNT: uart x receive DMA count(32bit addressing, Write Only).

Bit	Name	RW	Description
31-0	UTx_RXCNT	wo	写 UTx_RXCNT，控制器产生一次 DMA 的操作，同时清空中断，当 uart 需要接收的数据达到 UTx_RXCNT 的值，控制器会停止接收数据的操作，同时产生中断（UTx_CON[14]）。

9. UTx_RXSADR: uart x receive DMA address(25bit addressing, Write Only)

Bit	Name	RW	Description
24-0	UTx_RXSADR	wo	DMA 接收数据时，循环 buffer 的起始地址

10. UTx_RXEADR: uart x receive DMA end address(25bit addressing, Write Only).

Bit	Name	RW	Description
24-0	UTx_RXEADR	wo	DMA 接收数据时，循环 buffer 的结束地址

11. UTx_HRCNT: uart x have receive DMA count(32bit addressing, Read Only).

Bit	Name	RW	Description
31-0	UTx_HRCNT	ro	当设这 RDC（UTx_CON[7]）=1 时，串口设备会将当前总共收到的字节数记录到 UTx_HRCNT 里。

12. UTx_OTCNT: uart x OverTime count(32bit addressing, Write Only).

Bit	Name	RW	Description
31-0	UTx_OTCNT	wo	

设置串口设备在等待多久 RX 下降沿的时间，如果在所设置的时间里没收到 RX 的下降沿，则产生 OT 中断（OT_PND）

$$\text{Time(ot)} = \text{Time(uart_clk)} * \text{UTx_OTCNT};$$

例如：波特率时间为 100ns，UTx_OTCNT = 10,那么，OT 的时间就为 1000ns

【注意】：

1. OT_PND 触发流程，满足一下（1-4）顺序，则可产生 OT_PND：
 - 1)清 OT_PND;
 - 2)写 UTx_OTCNT;
 - 3)收到 n 个 byte 数据;
 - 4)从最后一个下降沿开始，等待 OT 超时;
 - 5)当 OTCNT 与超时时间相等，OT_PND 置 1。
2. OT_PND 不受 RPND 影响，即 RX_PND 置 1 后，后面如果没有再写一次 UTx_OTCNT，OT_PND 还是会起来;
3. 下降沿包括但不限于 start_bit，数据从 1 到 0 变化;
4. 如果芯片有 lsb_clk，uart_clk 就是 lsb_clk，否则看该芯片的时钟说明;

第 11 章. USB BRIDGE

11.1. 概述

负责控制系统与 USB 模块之间的通讯，包括：

- 1.接收 CPU 的命令，控制 SIE；
- 2.管理 endpoint mapping；
- 3.负责 SIE 和 memory 的通讯；
- 4.控制 USB PHY。

表 26-1 寄存器列表

寄存器列表		USB		
USB_CONx	USB_CON0			
EPx_CNT	EP0_CNT	EP1_CNT	EP2_CNT	EP3_CNT
	EP4_CNT			
EP0_ADR	EP0_ADR			
EPx_TADR	EP0_TADR	EP1_TADR	EP2_TADR	EP3_TADR
	EP4_TADR			
EPx_RADR	EP0_RADR	EP1_RADR	EP2_RADR	EP3_RADR
	EP4_RADR			
USB_IO_CONx	USB_IO_CON0			
USB_SIE	FADDR	POWER	INTRTX1	INTRTX2
	INTRRX1	INTRRX2	INTRUSB	INTRTX1E
	INTRTX2E	INTRRX1E	INTRRX2E	INTRUSBE
	FRAME1	FRAME2	DEVCTL	
USB_EP0	CSR0		COUNT0	
USB_EPX	TXMAXP	TXCSR1	TXCSR2	RXMAXP

	RXCSR1	RXCSR2	RXCOUNT1	RXCOUNT2
	TXTYPE	TXINTERVAL	RXTYPE	RXINTERVAL
TXDLY_CON				

表 26-2 Endpoint 大小

EP 号	BUFFER 大小(BYTE)
EP0	64
EP1 TX	1024
EP1 RX	1024
EP2_TX	1024
EP2 RX	1024
EP3 TX	1024
EP3 RX	1024
EP4 TX	1024
EP4 RX	1024

11.2. 寄存器说明

1.USB_CON0

Bit	Name	RW	Description
31-24	-	-	预留
23	EP4_DISABLE	rw	关闭端点 4, 不会返回 ack, nak, stall
22	EP3_DISABLE	rw	关闭端点 3, 不会返回 ack, nak, stall
21	EP2_DISABLE	rw	关闭端点 2, 不会返回 ack, nak, stall
20	EP1_DISABLE	rw	关闭端点 1, 不会返回 ack, nak, stall
19	RST_STL	rw	EP0 从 stall 状态复位为 idle 状态
13	SOF_PND	r	SOF 中断请求标志

12	CLR_SOFP	w	写 1 清除 SOF 中断请求标志，写 0 无效
11	SIEIE	rw	SIE 中断使能
10	SOFIE	rw	SOF 中断使能
9	PDCHKDP	rw	DP 外接下拉检查使能 0: disable 1: enable
8-7	-	-	预留
6	USB_TEST	rw	USB 测试模式
5	VBUS	rw	USB 电源
4	CID	rw	USB 工作模式: 0: host 1: device
3	TM1	rw	用于缩短检测连接时间(short connect timeout): 0: disable 1: enable
2	USB_NRST	rw	USB 模块复位: 0: reset 1: release reset
1	LOW_SPEED	rw	低速 USB_DMA 使能: 0: disable 1: enable
0	PHY_ON	rw	USB_PHY 使能: 0: disable 1: enable

2.EP0_CNT, EP1_CNT, EP2_CNT, EP3_CNT, EP4_CNT

Bit	Name	RW	Description
-----	------	----	-------------

31-0	EP(n)_CNT	w	usb endpoint 0-4 发送数据个数，单位为 byte
------	-----------	---	----------------------------------

3.EP0_ADR

Bit	Name	RW	Description
31-0	EP0_ADR	w	usb endpoint 0 发送/接收数据的起始地址

4.EP1_TADR, EP2_TADR, EP3_TADR, EP4_TADR

Bit	Name	RW	Description
31-0	EP(n)_TADR	w	usb endpoint 1-4 发送数据的起始地址

5.EP1_RADR, EP2_RADR, EP3_RADR, EP4_RADR

Bit	Name	RW	Description
31-0	EP(n)_RADR	w	usb endpoint 1-4 接收数据的起始地址

6.USB_IO_CON0

Bit	Name	RW	Description
31-15	-	-	预留
14	IO_PU_MODE	rw	IO 上下拉分两种模式 0: USB 模式 1: 普通 IO 模式
13	SR	rw	输出驱动能力选择 0: slow 1: fast
12	IO_MODE	rw	IO 模式使能 0: USB 模式 1: 普通 IO 模式
11	DMDIEH	rw	3.3V DM 数字输入使能

10	DPDIEH	rw	3.3V DP 数字输入使能
9	DMDIE	rw	1.2V DM 数字输入使能
8	DPDIE	rw	1.2V DP 数字输入使能
7	DMPD	rw	DM 下拉
6	DPPD	rw	DP 下拉
5	DMPU	rw	DM 上拉
4	DPPU	rw	DP 上拉
3	DMIE	rw	DM 方向 0: 输出 1: 输入
2	DPIE	rw	DP 方向 0: 输出 1: 输入
1	DMOUT	rw	DM 输出电平
0	DPOUT	rw	DP 输出电平

7.TXDLY_CON

Bit	Name	RW	Description
31-16	prd	rw	设置令牌包与数据包的延迟时钟个数
3-15	-	-	-
2	PND	r	延迟时钟数等于 PRD 时的标记位
1	CLR_PND	w	清除 pending
0	CLK_DIS	rw	1: 开启令牌包与数据包的延迟功能 0: 关闭令牌包与数据包的延迟功能

第 12 章. SD 控制器

12.1. 概述

- 1.SD 接口是一个标准的遵守 SD 协议的串行通讯接口，在上面传输的数据一 Byte(8bit)为最小单位。
- 2.SD 接口只支持主机模式。
- 3.SD 接口支持 1bit data 和 4bit data 模式：
 - 1bit data 模式：串行数据通过一根 DAT 线传输，一个字节数据需 8 个 SD 时钟。
 - 4bit data 模式：串行数据通过四根 DAT 线传输，一个字节数据需 2 个 SD 时钟。
- 4.SD 接口支持高速模式和低速模式，最高时钟速度可以达到 50MHz。
- 5.SD 接口支持命令 DMA 和数据 DMA，两个 DMA 时独立的，即可以同时做 DMA。

12.2. 控制寄存器

寄存器列表	SD0	SD1
SDx_CON0	SD0_CON0	SD1_CON0
SDx_CON1	SD0_CON1	SD1_CON1
SDx_CON2	SD0_CON2	SD1_CON2
SDx_CTU_CON	SD0_CTU_CON	SD1_CTU_CON
SDx_CTU_CNT	SD0_CTU_CNT	SD1_CTU_CNT
SDx_CPTR	SD0_CPTR	SD1_CPTR
SDx_DPTR	SD0_DPTR	SD1_DPTR

12.3. 模块寄存器

- 1.SDx_CON0: SD control register 0

Bit	Name	RW	Description
-----	------	----	-------------

15	DINT	r	DINT: SD 数据 pending 0:数据未结束 1:数据结束
14	CLR_DINT	w	CLR_DINT: 清空 SD 数据 pending (读为 0) 0:N/A 1:清空
13	LINE4	rw	LINE4: 4 线模式控制位 0:1 线模式 1:4 线模式
12	DATCLK8	rw	DATCLK8: SD 发送/接收数据时时钟延长 8 个 CLK 0:延长关闭 1:延长打开
11	DATCRC	r	DATCRC: 接收数据的 CRC 标志位 0:接收数据的响应 CRC 正常 1:接收数据出错, CRC 错误
10-8	SDDMODE	w	SDDMODE: SD 数据模式 (kickstart) 000: no operation 100: 发送数据, 不检查“busy”位 101: 发送数据, 检查“busy”位 110: 接收数据, 不检查“busy”位 111: 接收数据, 检查“busy”位
7	CINT	r	CINT: SD 命令 pending 0:命令未结束 1:命令结束
6	CLR_CINT	w	CLR_CINT: 清空 SD 命令 pending (读为 0) 0:N/A 1:清空

5	TIMEOUT	r	TIMEOUT: 命令响应超时标志位 0:命令响应正常 1:命令响应超时
4	CMDCLK8	rw	CMDCLK8: SD 发送/接收命令时钟延长 8 个 CLK 0:延长关闭 1:延长打开
3	CMDCRC	r	CMDCRC: 接收响应的 CRC 标志位 0:接收响应正确, CRC 正确 1:接收响应出错, CRC 错误
2-0	SDCMODE	w	SDCMODE: SD 命令模式(kickstart) 000:no operation 001:send command, receive 6-byte response, with check busy 010:send command, receive 17-byte response, with check busy 011:send command, without receive response, with check busy 101:send command, receive 6-byte response, without check busy 110:send command, receive 17-byte response, without check busy 111:send command, without receive response, without check busy 上面后六种命令模式设置后, sdc 控制器会进入命令收发状态

2.SDx_CON1: SD control register 1

Bit	Name	RW	Description
15-8	SD_BAUD	rw	SD_BAUD: SD 波特率 SD 波特率= $F_{sys}/(SD_BAUD+1)$
7	DATIE	rw	DATIE: SD 数据中断允许 0:不允许 1:允许

6	CMDIE	rw	CMDIE: SD 命令中断允许 0:不允许 1:允许
5	CLKALL	rw	CLKALL: SD 时钟开关 0:SD 时钟在不传送命令和数据时关闭 1:SD 时钟常开
4	reserved	r	保留
3-1	CRCSTA	r	CRCSTA: CRS State 详细请查阅 SD spec
0	SDCEN	rw	SDCEN: SD 控制器允许位 0:SD 控制器关闭 1:SD 控制器打开

3.SDx_CON2: SD control register 2

Bit	Name	RW	Description
15-9	reserved	r	保留
8-0	SDDCNT	rw	SDDCNT:数据读写数目 000:1byte 001:2byte 002:3byte . . 1ff:512byte

4.SDx_CTU_CON: SD data continue control register

Bit	Name	RW	Description
15-11	reserved	r	保留

10-8	SDDMODE_KEEP	rw	SDDMODE_KEEP: SD 连续数据读写模式（连续模式开始后会多次写进 SDDMODE, 不是 kickstart, kictstart 是写 SD_CTU_CNT） 000: N/A 100: 发送数据, 不检查“busy”位 101: 发送数据, 检查“busy”位 110: 接收数据, 不检查“busy”位 111: 接收数据, 检查“busy”位
7	CTU_PND	r	CTU_PND: SD 连续读写结束标志 0:未结束 1:结束
6	CLR_CTU_PND	w	CLR_CTU_PND: 清空 SD 连续读写结束标志（读为 0）（注意：先清 SD 数据 pending, 再清这里） 0:N/A 1:清空
5	CTU_ERR	r	CTU_ERR: SD 连续读写出错标志 0:正确 1:出错
4	CLR_CTU_ERR	w	CLR_CTU_ERR: 清空 SD 连续读写出错标志（读为 0）（注意：先清 SD 数据 pending, 再清这里） 0:N/A 1:清空
3	CTU_BUSY	r	CTU_BUSY: SD 数据连续读写状态 0:空闲 1:忙
2	CTU_PND_IE	rw	CTU_PND_IE: SD 读写结束中断允许 0:不允许 1:允许

1	CTU_ERR_IE	rw	CTU_ERR_IE: SD 读写出错中断允许（出错后会打断数据传输） 0:不允许 1:允许
0	SD_CTU_EN	rw	SD_CTU_EN: SD 数据多个 BLOCK 读写使能 0:关闭 1:打开

5.SDx_CTU_CNT: SD data continual transfer counter register

Bit	Name	RW	Description
15-0	SD_CTU_CNT	rw	SD 数据连续读写模式计数器（写寄存器 kictstart 数据传输）： 0: 开始 1 次 SD 数据读写 1: 开始 2 次 SD 数据读写 2: 开始 3 次 SD 数据读写 0xffff: 开始 65536 次 SD 数据读写 读寄存器可以看到剩下的数据传输次数： 0: 不再传输 1: 还剩 1 次传输 2: 还剩 2 次传输 0xffff: 还剩 65535 次传输

6.SDx_CPTR: SD command pointer

Bit	Name	RW	Description
17-0	SDCPTR	w	SD 命令地址 = SDCMDPTR

CMD responses 排列示例（CMD3）：

```
u32 __attribute__((aligned(16))) cmd_str[6];

u32 rca;

//send cmd

cmd_str[0] = ((0x40 + 3));

cmd_str[1] = 0x0;

JL_SD0->CPTR = (volatile u32)cmd_str;

.....

//obtain respond

sd0_cmd_finish_check();

rca = cmd_str[1] >> 24;

rca |= cmd_str[2] << 8;

rca &= 0x0000ffff;
```

表 27-1 CMD responses 排列示例表

cmd_str[0]	Cmd send byte 3	Cmd send byte 2	Cmd send byte 1	Cmd send byte 0
cmd_str[1]	Cmd res byte 1	Cmd res byte 0	Cmd send byte 5	Cmd send byte 4
cmd_str[2]	Cmd res byte 5	Cmd res byte 4	Cmd res byte 3	Cmd res byte 2
cmd_str[3]	Cmd res byte 9	Cmd res byte 8	Cmd res byte 7	Cmd res byte 6
cmd_str[4]	Cmd res byte 13	Cmd res byte 12	Cmd res byte 11	Cmd res byte 10
cmd_str[5]		Cmd res byte 16	Cmd res byte 15	Cmd res byte 14

（本质是接收响应的 memory 地址紧接在发送命令的 memory 地址后面）

7.SDx_DPTR: SD command pointer

Bit	Name	RW	Description
17-0	SDDPTR	w	SD 数据地址 = SDDATPTR

12.4. 配置流程示例

1. 发送命令、接收响应流程

1) 配置 SD_CON0 和 SD_CON1 进行初始化;

2) 往 memory 中写入长度为 40bit 的命令 (实际命令长度为 48bit, 其中 8bit 是 CRC 和结束位由硬件自动进行计算并补齐), 然后将数据的首地址配置给 SDCPTR。 (可将命令写入数组中, 然后将数组首地址配置给 SDCMDPTR, 见 2. SDCPTR 的 CMD responses 排列示例 (CMD3));

3) 配置 SDCMODE (SD_CON0[2:0]) 进行 kickstart, 开始发送命令;

4) 等待 PENDING (查询 CINT (SD_CON0[7])), 或等待中断到来;

5) 配置 CLR_CINT (SD_CON0[6]), 清除 PENDING;

6) 查询 CMDCRC (SD_CON0[3]), 确认接收的响应正确;

7) 查看接收的响应。

2. 发送数据流程 (单块写 CMD24)

1) 配置 SD_CON0 和 SD_CON1 进行初始化;

2) 配置 SD_CON2 进行配置将要发送数据的块长度;

3) 往 memory 中写入将要发送的数据, 然后将数据的首地址配置给 SDDPTR;

4) 配置 SDDMODE (SD_CON0[10:8]) 进行 kickstart, 开始发送数据;

5) 等待 PENDING (查询 DINT (SD_CON0[15])), 或等待中断到来;

6) 配置 CLR_DINT (SD_CON0[14]), 清除 PENDING;

7) 查询 CRCSTA (SD_CON1[3:1]), 确认发送的数据正确。

3. 接收数据流程 (单块读 CMD17)

1) 配置 SD_CON0 和 SD_CON1 进行初始化;

2) 配置 SD_CON2 进行配置将要接收数据的块长度;

3) 配置 SDDPTR 为数据保存在 memory 中的首地址;

4) 配置 SDDMODE (SD_CON0[10:8]) 进行 kickstart, 开始接收数据;

- 5) 等待 PENDING (查询 DINT (SD_CON0[15]))，或等待中断到来；
- 6) 配置 CLR_DINT (SD_CON0[14])，清除 PENDING；
- 7) 查询 DATCRC (SD_CON0[11])，确认接收的数据正确；
- 8) 查看接收的数据。

【注意】：进行单块读 CMD17 时，需先进行 kickstart 接收数据，然后发送命令，最后等待数据 PENDING，即在上述流程 4 和 5 之间进行发送命令。

4. 发送数据流程 (多块写 CMD25)

- 1) 配置 SD_CON0、SD_CON1 和 SD_CTU_CON 进行初始化；
- 2) 配置 SD_CON2 为将要发送数据的块长度，配置 SD_CTU_CNT 为将要发送数据的块数量；
- 3) 往 memory 中写入将要发送的数据，然后将数据的首地址配置给 SDDPTR；
- 4) 配置 SDDMODE_KEEP (SD_CTU_CON[10:8]) 进行 kickstart，开始发送数据；
- 5) 等待 PENDING (查询 CTU_PND (SD_CTU_CON[7]))，或等待中断到来；
- 6) 配置 CLR_CTU_PND (SD_CTU_CON[6])，清除 PENDING；
- 7) 查询 CTU_ERR (SD_CTU_CON[5])，确认发送数据过程没有发生错误 (发送各块数据返回的 CRC 状态有一个错误都会标志在该位)。

5. 接收数据流程 (多块写 CMD18)

- 1) 配置 SD_CON0、SD_CON1 和 SD_CTU_CON 进行初始化；
- 2) 配置 SD_CON2 为将要接收数据的块长度，配置 SD_CTU_CNT 为将要接收数据的块数量；
- 3) 配置 SDDPTR 为数据保存在 memory 中的首地址；
- 4) 配置 SDDMODE_KEEP (SD_CTU_CON[10:8]) 进行 kickstart，开始接收数据；
- 5) 等待 PENDING (查询 CTU_PND (SD_CTU_CON[7]))，或等待中断到来；
- 6) 配置 CLR_CTU_PND (SD_CTU_CON[6])，清除 PENDING；
- 7) 查询 CTU_ERR (SD_CTU_CON[5])，确认接收数据过程没有发生错误 (接收各块数据的 CRC 有一个错误都会标志在该位)；
- 8) 查看接收的数据。

【注意】：进行多块读 CMD18 时，需先进行 kickstart 接收数据，然后发送命令，最后等待数据 PENDING，即在上述流程 4 和 5 之间进行发送命令。

珠海市杰理科技股份有限公司

第 13 章. PWM_LED

13.1. 概述

PWM LED 是通过脉冲宽度调节控制 LED 的亮度与亮灭的一个模块。放在 1.2V SYSVDD，可以在软关机下工作。

PWM LED 模块可以配置两个周期大小不同的 PWM0、PWM1，分别控制 LED 的亮度（PWM0）与亮灭（PWM1）。

13.2. 特性

- 1.所有 IO 可用。
- 2.PDOWN（蓝牙 sniff）可用，SOFF 不可用。
- 3.目前只放了一个模块，只支持单 IO 单 LED、单 IO 双 LED。
- 4.驱动档位特性：

PWM 推高电平：高阻，上拉，输出 1 强驱 0，输出 1 强驱 1，输出 1 强驱 2，输出 1 强驱 3，保持，输出 1 强驱 2，输出 1 强驱 1，输出 1 强驱 0，上拉，高阻，保持（循环以上步骤）

PWM 推低电平：高阻，下拉，输出 0 强驱 0，输出 0 强驱 1，输出 0 强驱 2，输出 0 强驱 3，保持，输出 0 强驱 2，输出 0 强驱 1，输出 0 强驱 0，下拉，高阻，保持（循环以上步骤）

最高驱动档位可设，强驱及高阻保持时间可设，驱动切换等待时间可设（所有等待时间为同样的 n 个 PWM0_CLK）

- 5.推灯功能：

(1).固定亮度

（单 LED）单闪；双闪（双闪的两个时间长度可调，两个间隔可调）；

（双 LED）单闪 变色 单闪；

双闪 变色 双闪（双闪的两个时间长度可调，两个间隔可调）；

n 闪 变色 n 闪（n<7）；

(2).呼吸灯

（单 LED）灭灯时间长度可调，亮灯时间长度可调，亮灯快慢可调，灭灯快慢可调；

（双LED）（灭灯时间长度可调，亮灯快慢可调，灭灯快慢可调）呼吸 变色 呼吸；

无灯---A 灯逐渐变到最亮---最亮变色---B 灯由最亮逐渐变到最暗---变色（无灯）---A 灯逐渐变到最亮...；

无灯---A 灯呼吸---无灯---A 灯逐渐变到 最亮---最亮变色---B 灯由最亮逐渐变到最暗---无灯---B 灯呼吸---变色（无灯）---A 灯呼吸...；

13.3. 寄存器并接

1.PWM_PRD_DIV : pwm1_clk 分频比控制器，12bit

$PWM_PRD_DIV[11:0] = \{P3_PWM_CON3[3:0], PWM_PRD_DIVL[7:0]\};$

2.PWM_BRI_PRD[9:0]: 亮度周期，10bit

$PWM_BRI_PRD[9:0] = \{P3_PWM_BRI_PRDH[1:0], P3_PWM_BRI_PRDL[7:0]\};$

3.P3_PWM_BRI_DUTY0[9:0]: 推高电平点亮的灯，亮度占空比，10bit

$P3_PWM_BRI_DUTY0[9:0] = \{P3_PWM_BRI_PRD0H[1:0], P3_PWM_BRI_PRD0L[7:0]\};$

4.P3_PWM_BRI_DUTY1[9:0]: 推低电平点亮的灯，亮度占空比，10bit

$P3_PWM_BRI_DUTY1[9:0] = \{P3_PWM_BRI_PRD1H[1:0], P3_PWM_BRI_PRD1L[7:0]\};$

5.呼吸灯，灭灯延时计数器，16bit

$\{PWM_DUTY3[7:0], PWM_DUTY2[7:0]\};$

13.4. 时钟控制

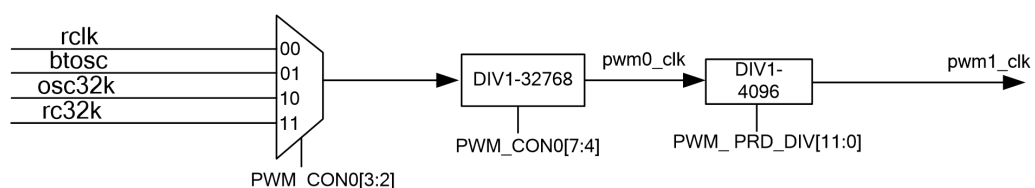


图 15-1 时钟控制框图

其中 PWM0 由 pwm0_clk 驱动，周期 PRD0 由 PWM_BRI_PRD0/1 10bit 决定，有一个循环计数 bright_cnt 决定显示亮度。

其中 PWM1 由 pwm1_clk 驱动，周期 PRD1 最多 256 个时钟，有一个循环计数 counter_cnt。

特别地，当呼吸灯功能打开，PWM0、PWM1 分别控制 LED 的亮度级数变化及最亮和最暗保持的时间。

13.5. 二级亮灭控制

PWM1 可以控制 LED 在一个 PRD1 周期亮灭一次或两次。

下图以一个 LED 灯 PWM 电平举例：

输出高电平亮

单周期双闪

单灯

非呼吸灯模式

配置：

PWM_DUTY3_EN = 0;

PWM_DUTY2_EN = 1;

PWM_DUTY1_EN = 1;

PWM_DUTY0_EN = 1;

PWM1_PRD_SEL = 0; //PWM1 PRD 为 256 pwm1_clk

PWM1_INV = 0; //固定为 0 不变，亮灭不取反，

如下图，周期开始时是从灯灭开始

PWM0_INV = 0; //固定为 0 不变，灯亮度控制

OUT_LOGIC = 1; //固定为 1 不变，PWM 输出逻辑控制

BREATHE_EN = 0;

SHIFT_DUTY = 0;

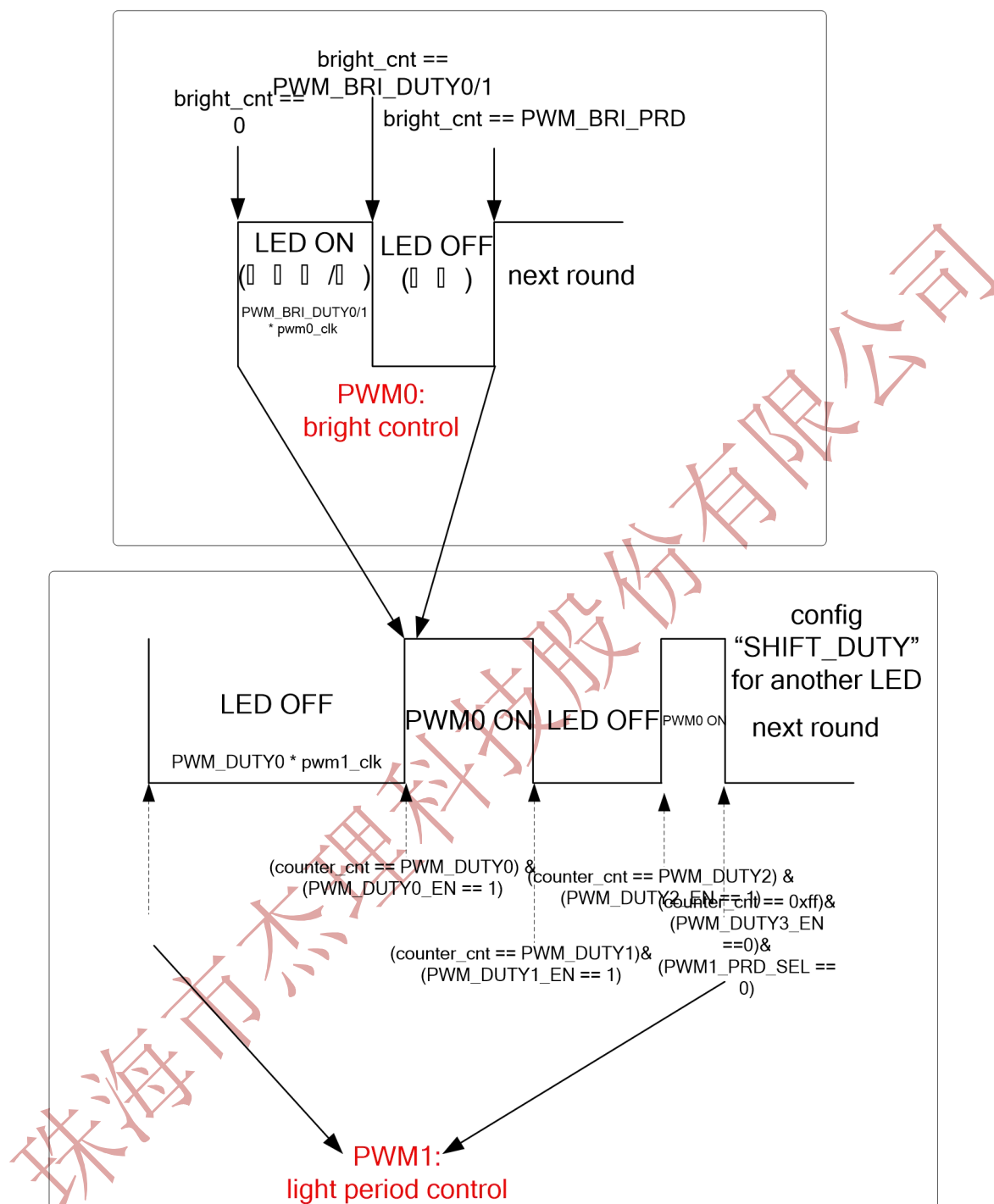


图 15-2 二级亮灭控制框图

【注意】：需要同时推两个颜色的灯时，请把 PWM1 的周期设置得尽可能短，小于人眼分辨极限即可。

13.6. 呼吸灯控制

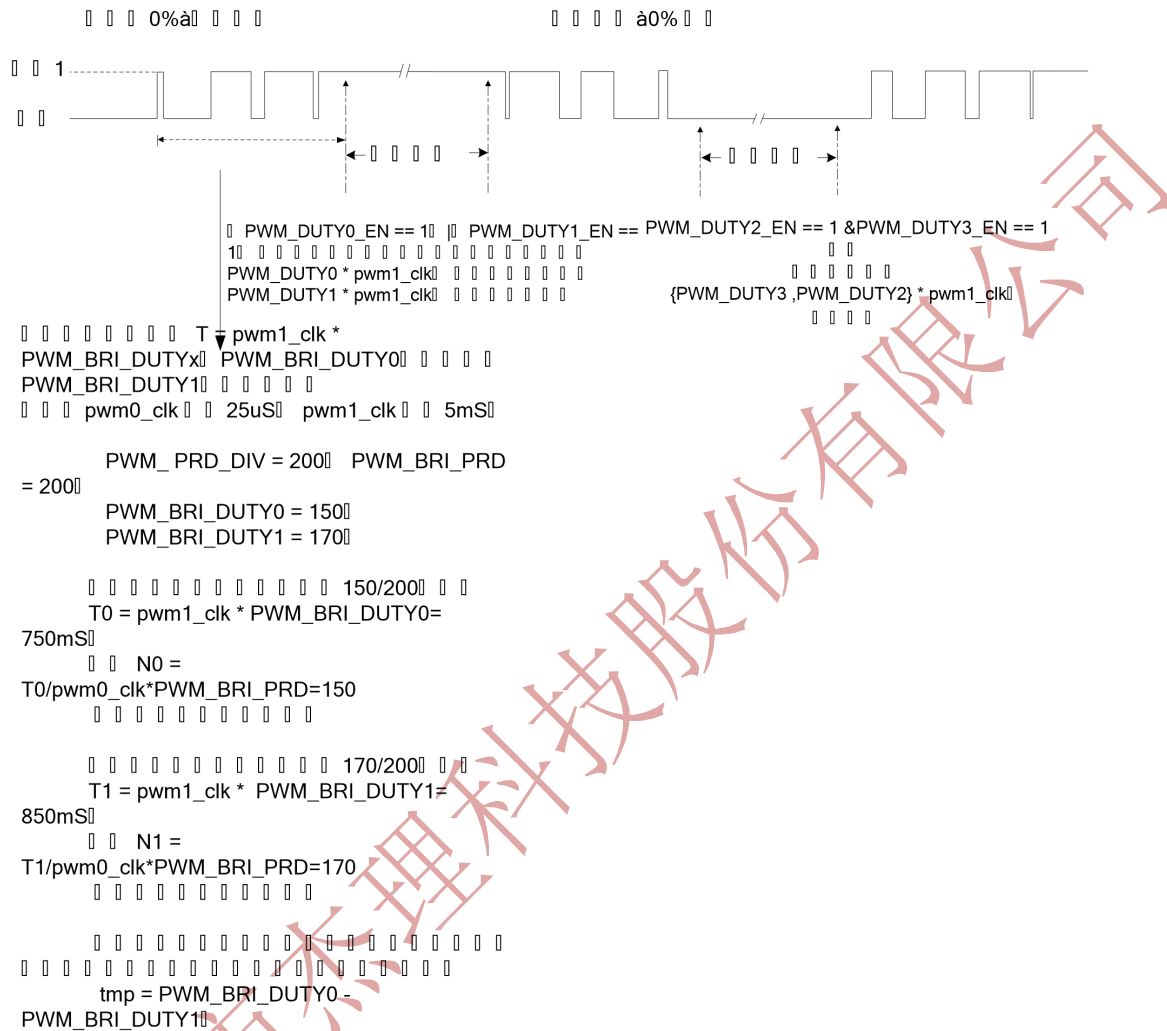


图 15-3 呼吸灯控制图

13.7. 3.3v 系统控制寄存器说明

1.PWM_CON0: PWM control register 0 (8bit addressing)

Bit	Name	RW	Description
7-4	PWM_PSET	rw	PWM_PSET:PWM pre_scaler setting。设置 PWM0 的分频器除法因子 [1:0]: 00:div101:div4 10:div16 11:div64 [2]:0: x11: x2 [3]:0: x11: x256 计算方式: DIV = [1:0]的 divx 乘以 [2]倍数 乘以 [3]倍数。
3-2	PWM_SSEL	rw	PWM_SSEL:PWM 时钟源的选择。 00: rclk (有偏差 250KHz) 01: btosc (准确 12/24MHz) 10: osc32k (准确 32KHz) 11: lrc32k (需校准 32KHz)
1	BREATHE_EN	rw	呼吸灯使能。 0: 呼吸灯功能关闭 1: 呼吸灯功能打开
0	PWM_EN	rw	PWM 使能。 0: 模块总使能关闭 1: 模块总使能打开

2.PWM_CON1: PWM control register 1 (8bit addressing)

Bit	Name	RW	Description
7	PWM_DUTY3_EN	rw	亮灭 DUTY 控制 3 开关/呼吸灯空周期延时开关高 8bit 0: 亮灭 DUTY 控制 3 关闭 (BREATHE_EN == 0 &

			PWM1_PRD_SEL == 0) / 呼吸灯空周期延时关闭 (BREATHE_EN == 1) 1: 亮灭 DUTY 控制 3 开启 (BREATHE_EN == 0 & PWM1_PRD_SEL == 0) / 呼吸灯空周期延时开启 (BREATHE_EN == 1)
6	PWM_DUTY2_EN	rw	亮灭 DUTY 控制 2 开关/呼吸灯空周期延时开关低 8bit 0: 亮灭 DUTY 控制 2 关闭 (BREATHE_EN == 0) / 呼吸灯满周期延时关闭 (BREATHE_EN == 1) 1: 亮灭 DUTY 控制 2 开启 (BREATHE_EN == 0) / 呼吸灯满周期延时开启 (BREATHE_EN == 1)
5	PWM_DUTY1_EN	rw	亮灭 DUTY 控制 1 开关/呼吸灯最高亮度延时开关(高电平灯) 0: 亮灭 DUTY 控制 1 关闭 (BREATHE_EN == 0) / 呼吸灯空周期延时关闭 (BREATHE_EN == 1) 1: 亮灭 DUTY 控制 1 开启 (BREATHE_EN == 0) / 呼吸灯空周期延时开启 (BREATHE_EN == 1)
4	PWM_DUTY0_EN	rw	PWM_DUTY0_EN:亮灭 DUTY 控制 0 开关/呼吸灯最高亮度延时开关 0: 亮灭 DUTY 控制 0 关闭 (BREATHE_EN == 0) / 呼吸灯满周期延时关闭 (BREATHE_EN == 1) 1: 亮灭 DUTY 控制 0 开启 (BREATHE_EN == 0) / 呼吸灯满周期延时开启 (BREATHE_EN == 1)
3	PWM1_PRD_SEL	rw	PWM1 亮灭周期选择: 0: PWM1 周期为 0xff 1: PWM1 周期为 PWM_DUTY3
2	PWM_OUTPUT_INV	rw	输出电平取反 (周期变色 SHIFT_DUTY==0 时可用) 0: 不取反。 1: 取反

1	PWM1_INV	rw	PWM1 输出的 LED 亮灭波形取反（默认由 0 开始），可以控制亮灭周期是从灯亮开始还是灯灭开始，软件常设为 0。 0: PWM1 输出的 LED 亮度波形不取反（灭灯开始的 PWM1 周期） 1: PWM1 输出的 LED 亮度波形取反（亮灯开始的 PWM1 周期）
0	PWM0_INV	rw	PWM0 输出的 LED 亮度波形取反（默认由 1 开始），软件常设为 0 0: PWM0 输出的 LED 亮度波形不取反 1: PWM0 输出的 LED 亮度波形取反

3.PWM_CON2: PWM control register 2 (8bit addressing)

Bit	Name	RW	Description
7-4	SHIFT_DUTY	rw	LED 周期变色（输出电平取反），非呼吸灯 0000: 不变色 0001: 1 个 PWM1 周期变色 ... 1111: 15 个 PWM1 周期变色 LED 周期变色，呼吸灯 PWM_CON2[4]常设为 0 0000: 不变色 0010: 1 个周期变色（1 个周期为：最暗到最亮，再到最暗） 0100: 2 个周期变色 ... 1110: 7 个周期变色
3-2	IO_DRV_SHIFT_CLK	rw	输出驱动切换时钟个数（PWM0_CLK） 00: 1 01: 2 10: 4

			11: 8
1-0	IO_MAX_DRV	rw	IO 最大输出强度 00: 2.4mA (8mA mos + 120Ω res) 01: 8mA (8mA mos) 10: 18.4mA (24mA mos + 120Ω res) 11: 24mA (24mA mos)

4.PWM_CON3: PWM control register 3 (8bit addressing)

Bit	Name	RW	Description
7	PENDING	r	亮灭周期结束，呼吸灯周期结束，pending 置 1
6	CLEAR PENDING	w	清 pending 0: 无效 1: 清 pending
5	IE	rw	PWM LED pending 中断使能。
4	OUT_LOGIC	rw	输出逻辑， 软件常设为 1 0: LED 在低电平时为亮 (PWM0 PWM1)。 1: LED 在高电平时为亮 (PWM0 & PWM1)。
3-0	PWM_PRD_DIVH	rw	pwm1 clk 分频比控制高位 PWM_PRD_DIV[11:8]

5.PWM_BRI_PRDL: PWM0 brightness period register (8bit addressing)

Bit	Name	RW	Description
7-0	PWM_BRI_PRD[7:0]	rw	亮度周期低 7bit

6.PWM_BRI_PRDH: PWM0 brightness period register (8bit addressing)

Bit	Name	RW	Description
1-0	PWM_BRI_PRD[9:8]	rw	亮度周期高 2bit

7.PWM_BRI_DUTY0L: PWM0 brightness duty register (8bit addressing)

Bit	Name	RW	Description
7-0	PWM_BRI_DUTY0 [7:0]	rw	亮度控制低 7bit, 例: 高电平灯亮 PWM_BRI_DUTY0 = 0; 灯最暗 PWM_BRI_DUTY0 = 1; 灯亮度增强一档 ... PWM_BRI_DUTY0 = PWM_BRI_PRD; 灯最亮

8.PWM_BRI_DUTY0H: PWM0 brightness duty register (8bit addressing)

Bit	Name	RW	Description
7-0	PWM_BRI_DUTY0 [9:8]	rw	亮度控制高 2bit, 例: 高电平灯亮

9.PWM_BRI_DUTY1L: PWM0 brightness duty register (8bit addressing)

Bit	Name	RW	Description
7-0	PWM_BRI_DUTY1 [7:0]	rw	亮度控制低 7bit, 例: 低电平灯亮 PWM_BRI_DUTY0 = 0; 灯最暗 PWM_BRI_DUTY0 = 1; 灯亮度增强一档 ... PWM_BRI_DUTY0 = PWM_BRI_PRD; 灯最亮

10.PWM_BRI_DUTY1H: PWM0 brightness duty register (8bit addressing)

Bit	Name	RW	Description
7-0	PWM_BRI_DUTY1 [9:8]	rw	亮度控制高 2bit, 例: 低电平灯亮

11.PWM_PRD_DIVL: PWM1 period divider register (8bit addressing)

Bit	Name	RW	Description
7-0	PWM_PRD_DIV[7:0]	rw	PWM 时钟的分频因子设置。

PWM_PRD_DIV : pwm1_clk 分频比控制器, 12bit

PWM_PRD_DIV[11:0] = {P3_PWM_CON3[3:0],PWM_PRD_DIVL[7:0]};

0x000:div = 1;

0x001:div = 2;

... ..

0xfff:div = 4096;

12.PWM_DUTY0: PWM duty0 register (8bit addressing)

Bit	Name	RW	Description
7-0	PWM_DUTY0	rw	BREATHE_EN == 0 & PWM_DUTY0_EN == 1: 亮灭 DUTY 控制 1 BREATHE_EN == 1 & PWM_DUTY0_EN == 1: 呼吸灯最高亮度延时(低电平灯)

13.PWM_DUTY1: PWM duty1 register (8bit addressing)

Bit	Name	RW	Description
7-0	PWM_DUTY1	rw	BREATHE_EN == 0 & PWM_DUTY1_EN == 1: 亮灭 DUTY 控制 1 BREATHE_EN == 1 & PWM_DUTY1_EN == 1: 呼吸灯最高亮度延时(高电平灯)

14.PWM_DUTY2: PWM duty2 register (8bit addressing)

Bit	Name	RW	Description
-----	------	----	-------------

7-0	PWM_DUTY2	rw	BREATHE_EN == 0 & PWM_DUTY2_EN == 1: 亮灭 DUTY 控制 2 BREATHE_EN == 1 & PWM_DUTY2_EN == 1: 呼吸灯灭灯延时低 8bit
-----	-----------	----	--

15.PWM_DUTY3: PWM duty3 register (8bit addressing)

Bit	Name	RW	Description
7-0	PWM_DUTY3	rw	BREATHE_EN == 0 & PWM1_PRD_SEL == 0 & PWM_DUTY3_EN == 1: 亮灭 DUTY 控制 3 BREATHE_EN == 0 & PWM1_PRD_SEL == 1: 亮灭 PWM1 周期 PWM1_PRD BREATHE_EN == 1 & PWM_DUTY3_EN == 1: 呼吸灯灭灯延时高 8bit

13.8. 软件 DEMO

```
//-----//
```

```
// led define for breathe demo
```

```
//-----//
```

```
#define led_con0_init
```

```
/* PWM0 PDIV
```

```
4bit RW */( 1<<4 ) | \
```

```
/* CLKSEL
```

```
2bit RW */( 3<<2 ) /* 0:RC250k 1:BTOSC 2:OSL 3:LRC */ | \
```

```
/* BREATHE
```

```
1bit RW */( 1<<1 ) | \
```

```
/* EN
```

```
1bit RW */( 1<<0 )
```

```
//PWM0 PDIV
```

```
//pre-scaler setting,
```

```
[1:0]: 00:div1
```

```
01:div4
```

```
10:div16
```

11:div64

```

//
//
// [2]: 0: x1 1: x2
// [3]: 0: x1 1: x256

#define led_con1_init
    /* PWM_DUTY3_EN 1bit RW */ ( 1<<7 ) | \
    /* PWM_DUTY2_EN 1bit RW */ ( 1<<6 ) | \
    /* PWM_DUTY1_EN 1bit RW */ ( 1<<5 ) | \
    /* PWM_DUTY0_EN 1bit RW */ ( 1<<4 ) | \
    /* PWM1_PRD_SEL 1bit RW */ ( 0<<3 ) /* 0 pwm1 round end by 0xff; 1 pwm1
round end by pwm_duty3 */ | \
    /* PWM_OUT_INV 1bit RW */ ( 0<<2 ) /* using in shift == 0 */ | \
    /* PWM_EDGE1 1bit RW */ ( 0<<1 ) | \
    /* PWM_EDGE0 1bit RW */ ( 0<<0 )

#define led_con2_init
    /* SHIFT PRD 4bit RW */ ( 2<<4 ) | \
    /* DRIVING CYCLE 2bit RW */ ( 2<<2 ) | \
    /* DRIVING MAX 2bit RW */ ( 2<<0 )

#define led_con3_init
    /* PWM PND 1bit RO */ ( 0<<7 ) | \
    /* PWM CLR PND 1bit WO */ ( 1<<6 ) | \
    /* PWM IE 1bit RW */ ( 1<<5 ) | \
    /* OUT_LOGIC 1bit RW */ ( 1<<4 ) | \
    /* PWM1 DIVH 4bit RW */ ( 2<<0 )

void led_init (void)

```

```
{  
  
    p33_tx_1byte(P3_LRC_CON0, 0x61);  
    p33_tx_1byte(P3_LRC_CON1, 0xaa);  
  
    p33_tx_1byte(P3_PWM_BRI_PRDL, 0x40);  
    p33_tx_1byte(P3_PWM_BRI_PRDH, 0x01);  
    p33_tx_1byte(P3_PWM_BRI_DUTY0L, 0x71);  
    p33_tx_1byte(P3_PWM_BRI_DUTY0H, 0x00);  
    p33_tx_1byte(P3_PWM_BRI_DUTY1L, 0x90);  
    p33_tx_1byte(P3_PWM_BRI_DUTY1H, 0x00);  
  
    p33_tx_1byte(P3_PWM_PRD_DIVL, 0x40);  
  
    p33_tx_1byte(P3_PWM_DUTY0, 0x01);  
    p33_tx_1byte(P3_PWM_DUTY1, 0x09);  
    p33_tx_1byte(P3_PWM_DUTY2, 0xb0);  
    p33_tx_1byte(P3_PWM_DUTY3, 0x02);  
  
    p33_tx_1byte(P3_PWM_CON3, led_con3_init);  
    p33_tx_1byte(P3_PWM_CON2, led_con2_init);  
    p33_tx_1byte(P3_PWM_CON1, led_con1_init);  
    p33_tx_1byte(P3_PWM_CON0, led_con0_init);  
    p33_or_1byte(P3_PWM_CON3, BIT(6)); //clear pending  
}
```